Introduction to Machine Learning and Data analysis

Data Visualization

Mahdi Shafiee Kamalabad,and Maarten Cruyff

Content

Data visualization

1. Exploratory data analysis
2. Tufte's Principles of Graphical Excellence
3. Additional Advice: *Do*'s and *Don't*s
4. Grammar of Graphics
5. Some examples
6. Lab preview

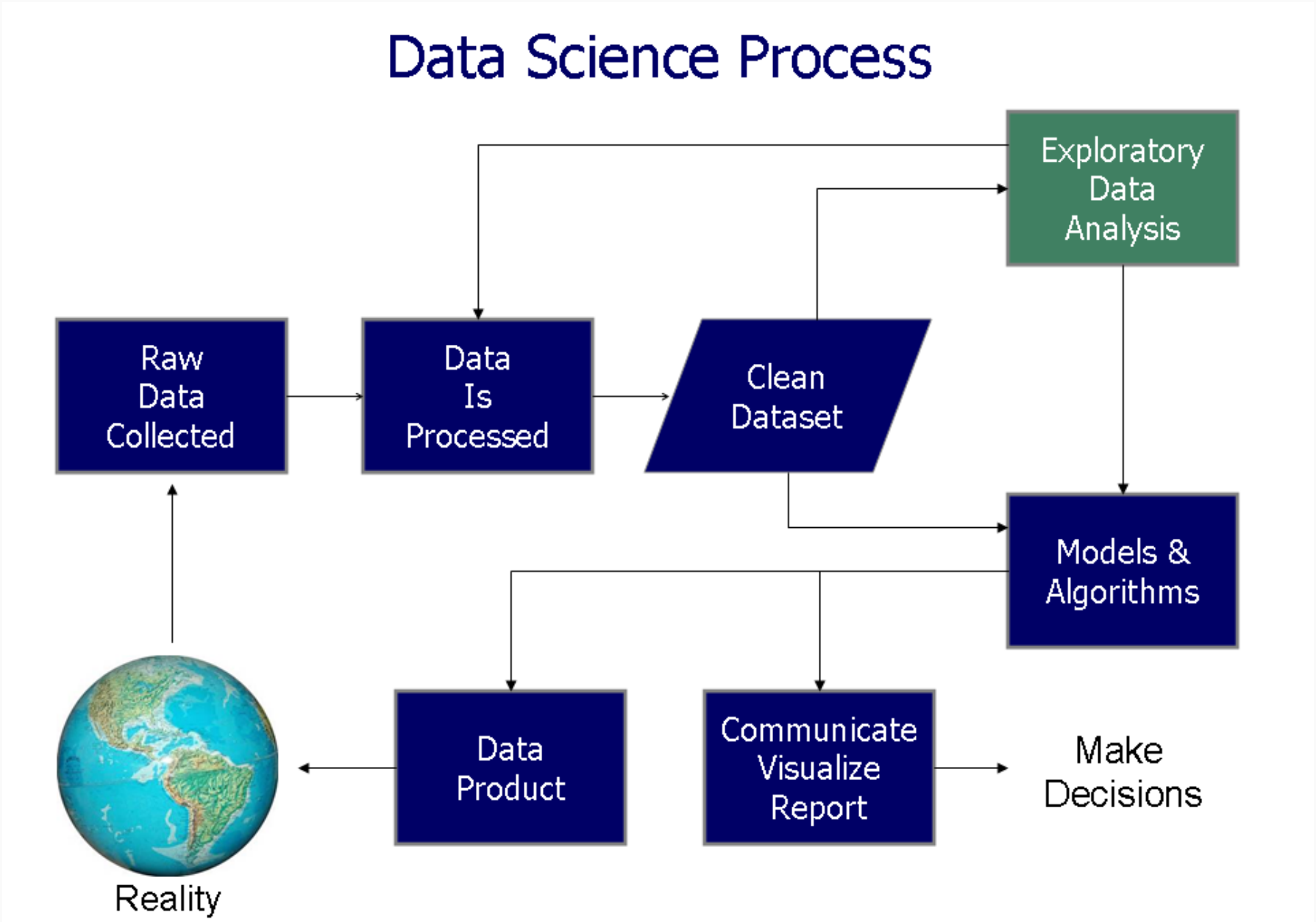
What's data visualization?

Communication of data by encoding it as visual objects, i.e.

- dots, lines, bars, etc.

to make data more accessible, understandable and usable.

Integral part of data science process



Data visualization in data science

Data visualization can be used at several stages, not only at the end:

- during data cleaning, to spot errors or missing values;
- during exploratory analysis, to find patterns and detect outliers and other anomalies
- during modeling, to evaluate results;
- during communication, to explain findings.

Early Example: John Snow's Cholera Map

The 1854 Soho cholera outbreak was not due to 'miasma' (John Snow)

So, a good visualisation can make hidden patterns and insights much easier to see.

Tufte's Principles of Graphical Excellence

Guidelines for representing visual information

"Graphical excellence is that which gives to the viewer the greatest number of ideas in the shortest time with the least ink in the smallest space."

Edward R. Tufte, The Visual Display of Quantitative Information

- Proportionality principle
- Maximize data-ink ratio
- Omit chart junk

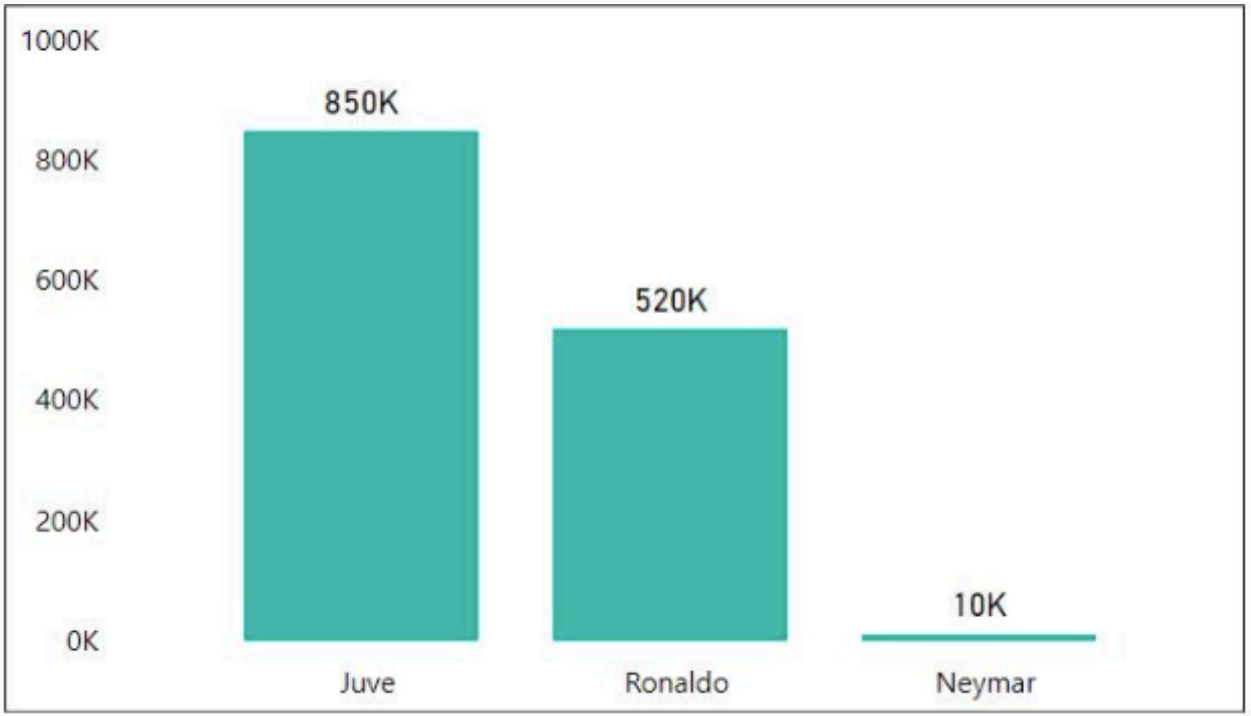
See also Anil Bas' slides on Tufte:
https://anilbas.github.io/teaching/hci/week13/Tufte.pdf

Proportionality principle

- Visual elements should represent data accurately and not distort the true values.

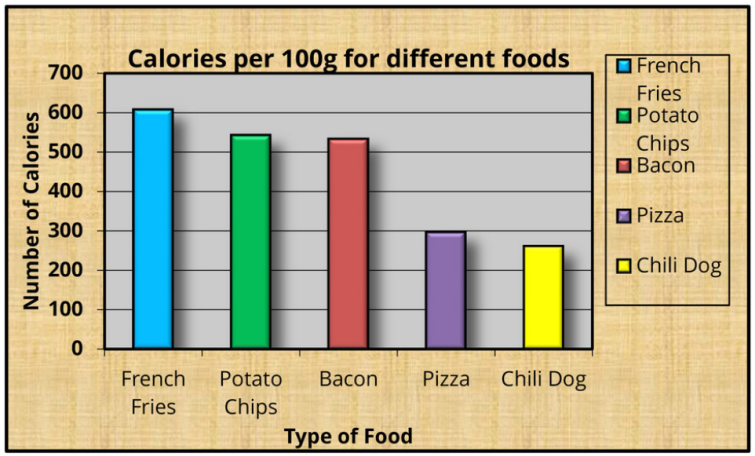
Proportionality principle

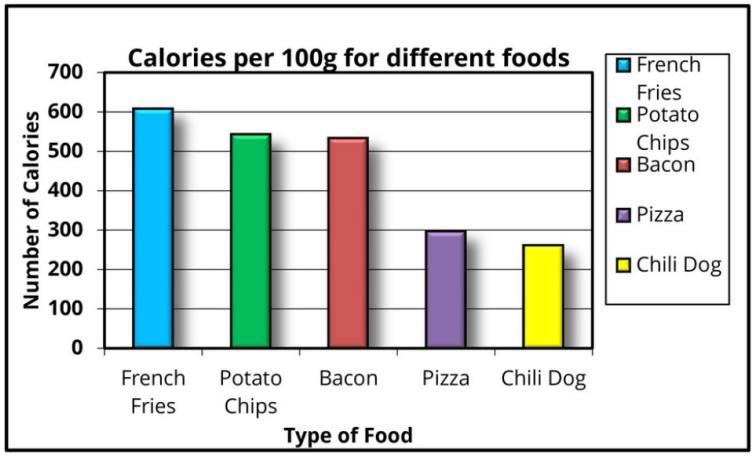
- Visual elements should represent data accurately and not distort the true values.

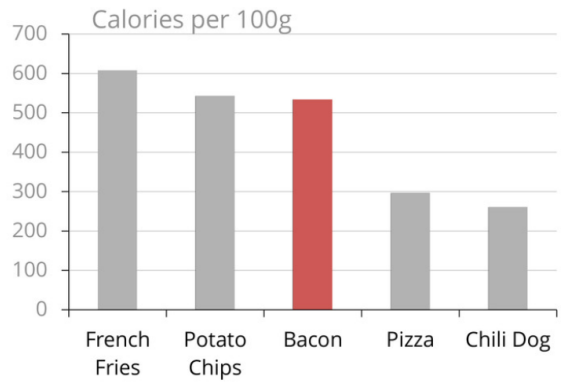


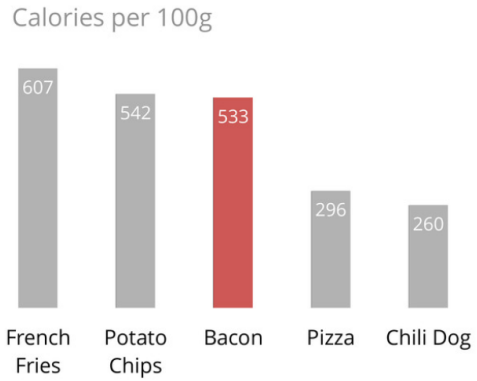
Maximize data-to-ink ratio

- How much of the graphic shows data versus non-data elements.
- Try using color and text separately.
  https://darkhorsevisualization.com/blog/data-looks-better-naked







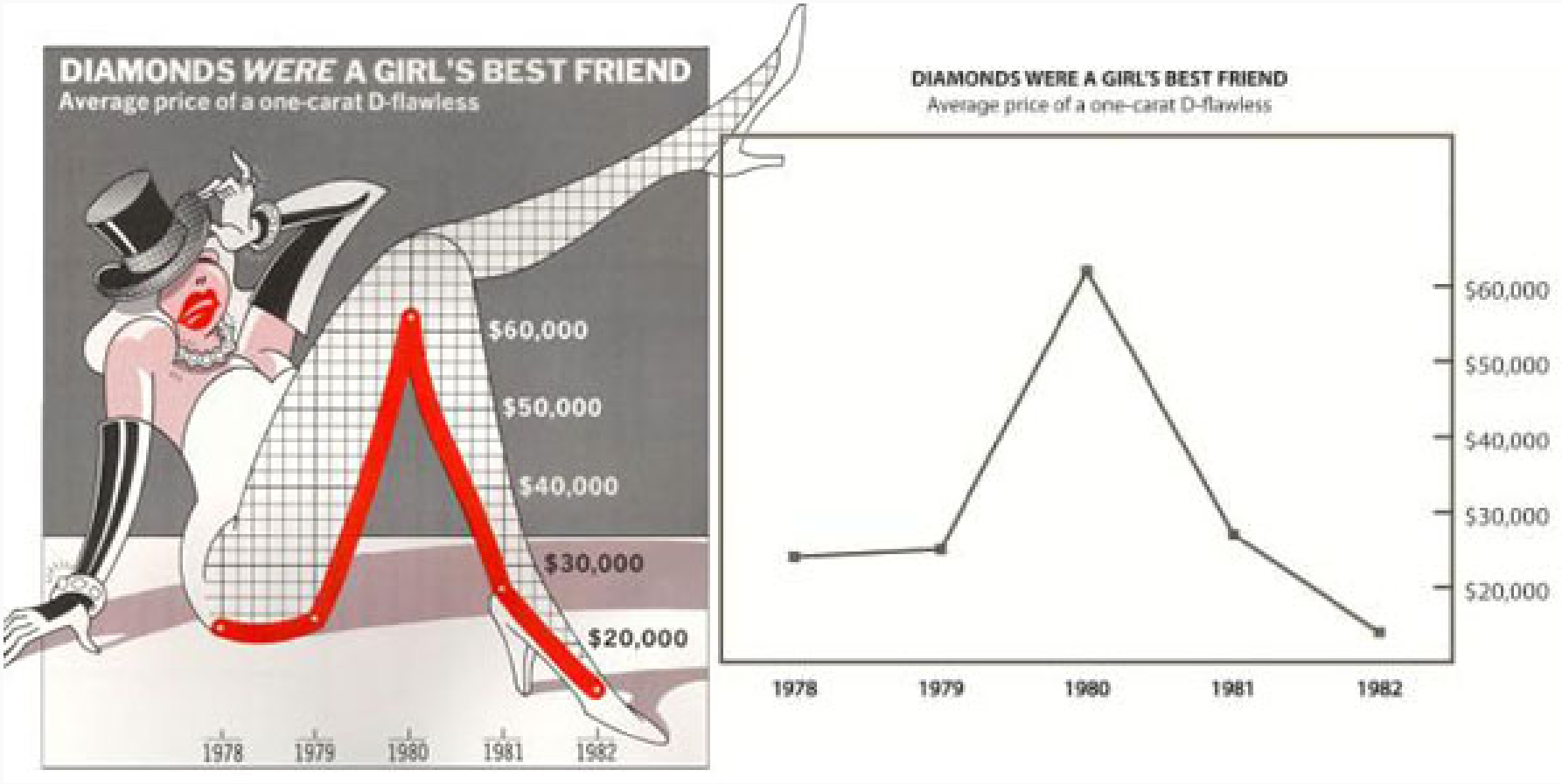


Omit chartjunk

Chartjunk includes all forms of unnecessary decoration or clutter, such as:

- 3D effects
- Shadows and gradients
- Bright or distracting colors
- Background textures or images
- Thick borders
- Decorative icons
- Extra gridlines
- Overly large labels or legends

Omit chart junk

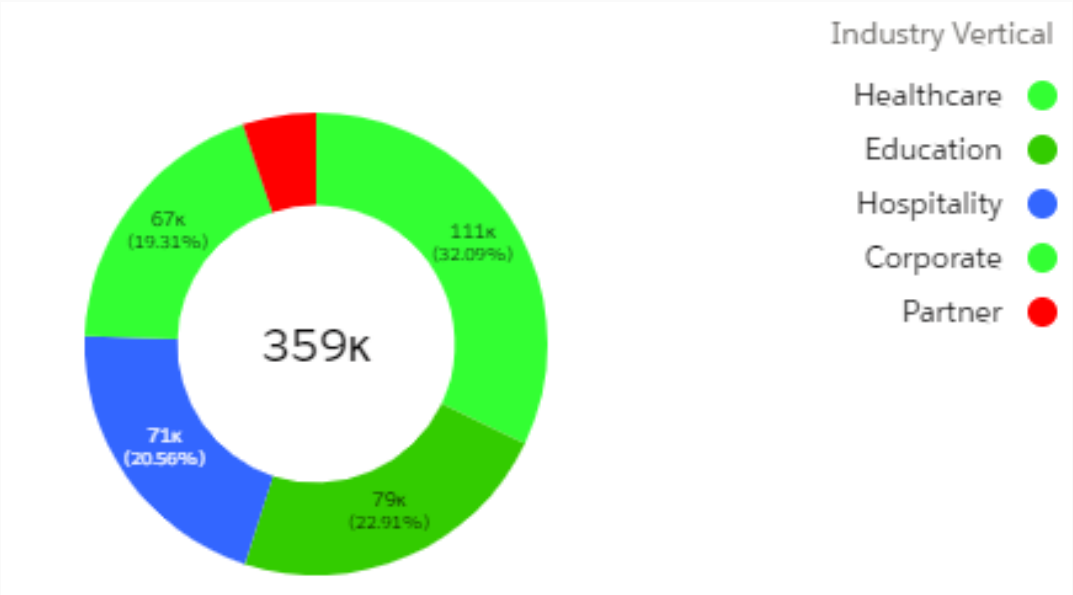


Avoid pie charts

Pie charts are the worst as they are often difficult to read because:

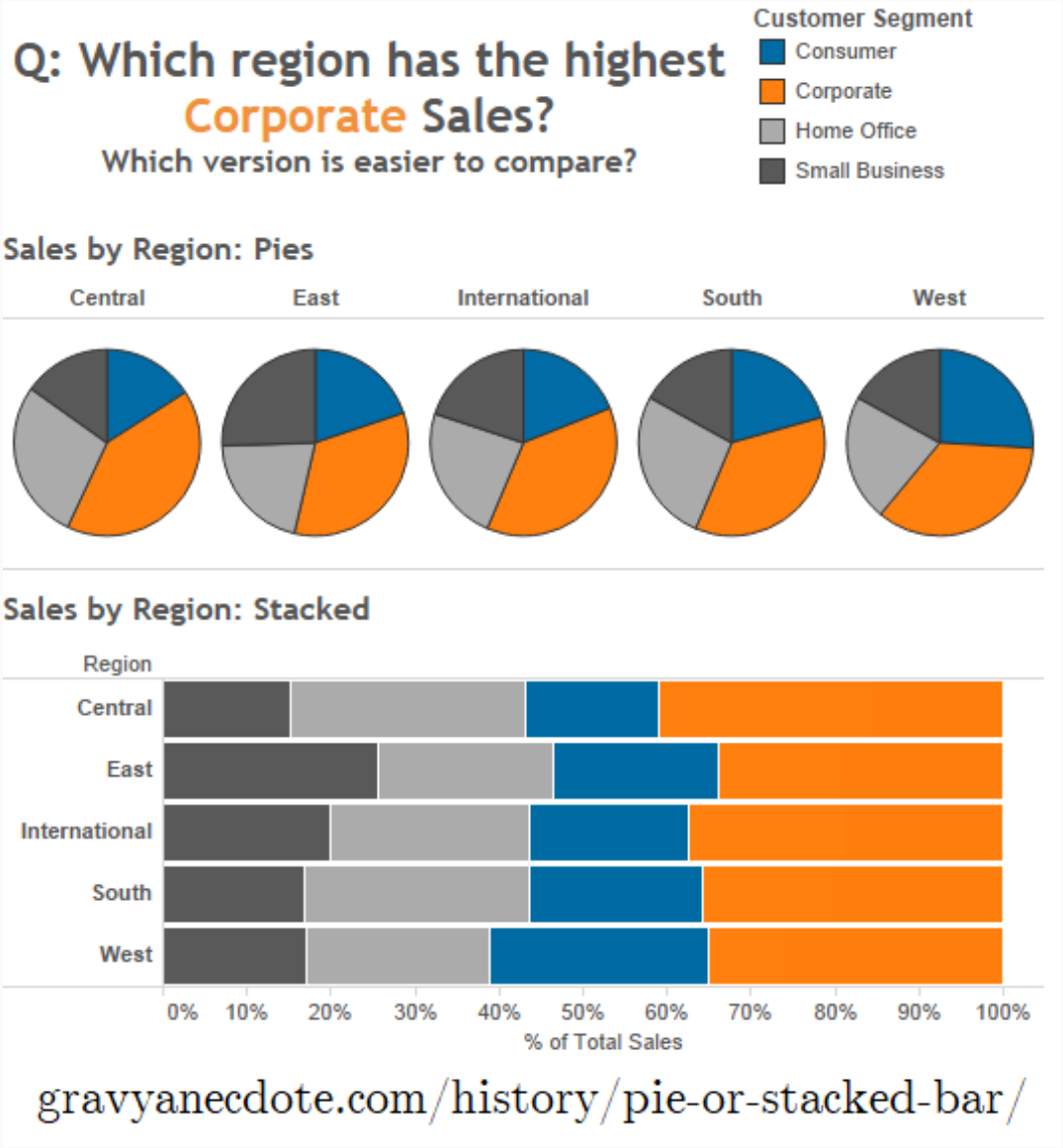
- Slice angles and labels are hard to compare.
- Slice order, orientation, and color can influence perception.
- Comparing several pie charts is especially difficult.

You can instead use a bar chart when accurate comparison is important.



"The only thing worse than a pie chart is several of them."

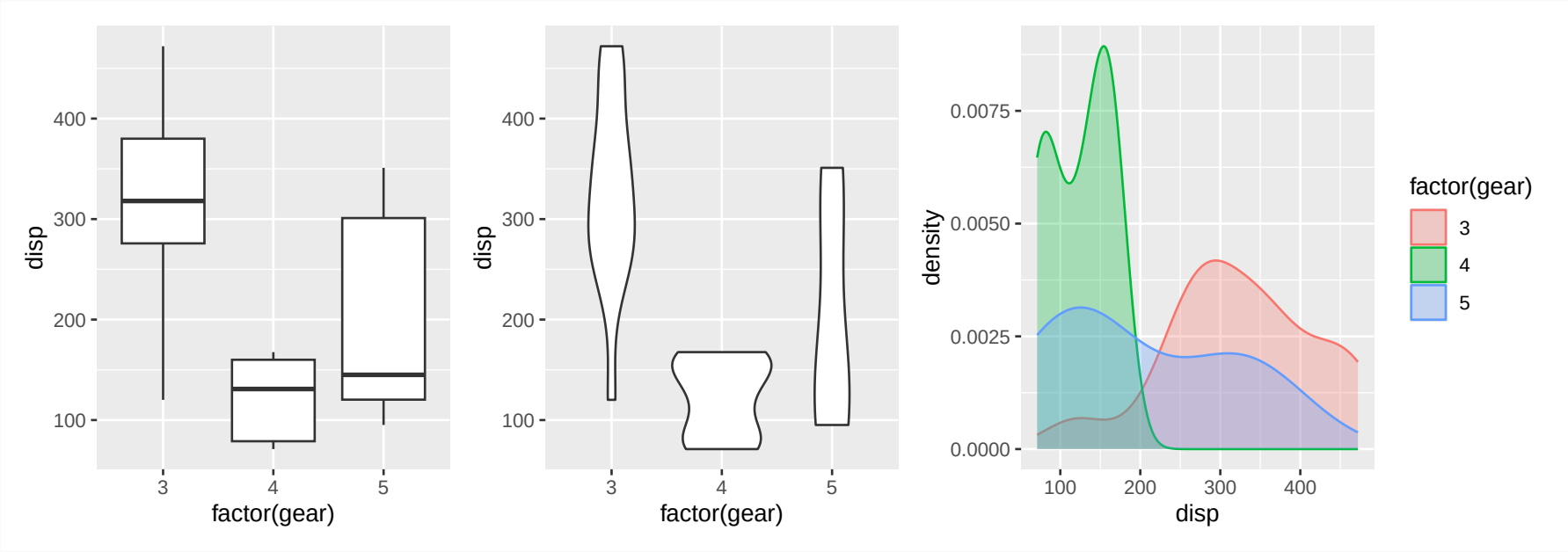
Replace pie charts with (stacked) bar charts



Box plots

Box plots are crude and may miss relevant information.

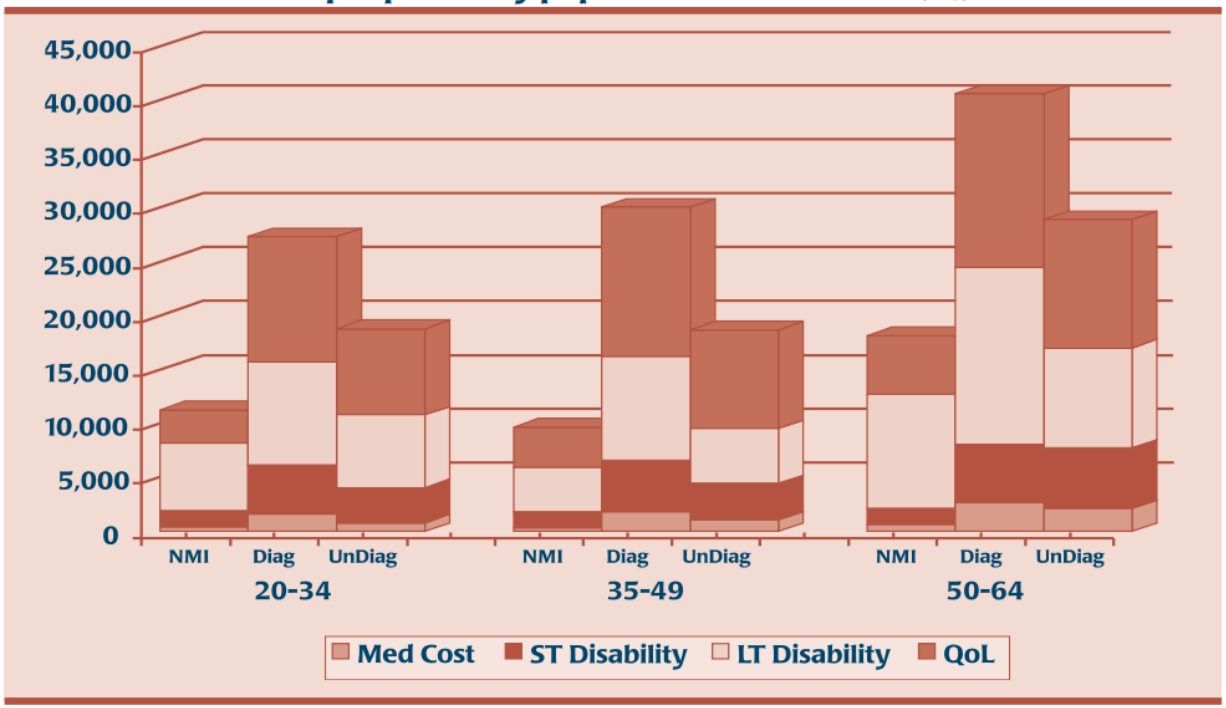
- good alternatives are violin and density plots



Avoid irrelevant dimensions

Do not plot two dimensional data in 3D

FIGURE 2
Cost per person by population characteristic (C$)

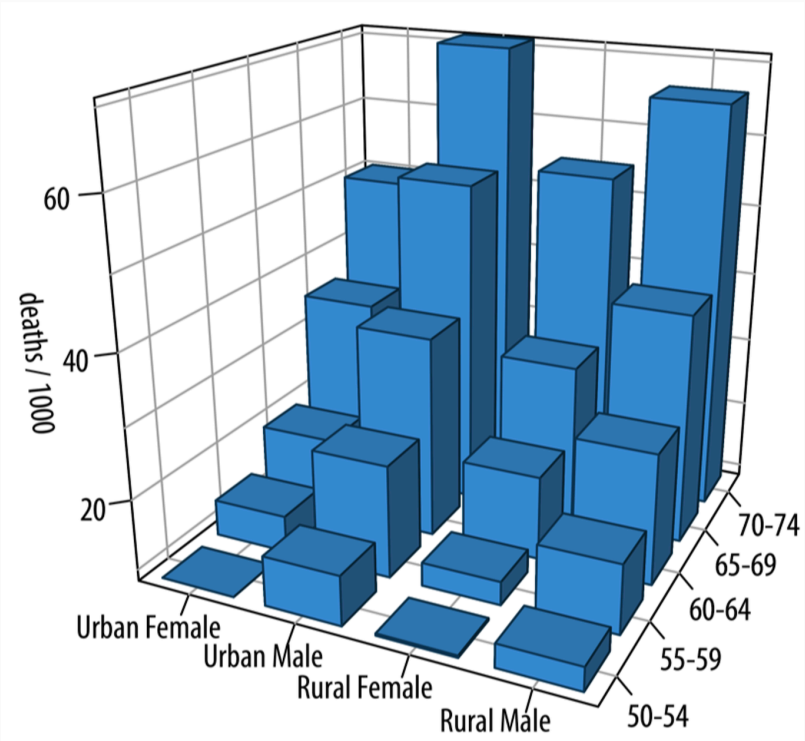


N MI = no mental illness, Diag = diagnosed mental illness, Un Diag = undiagnosed mental illness, Med Cost = direct medical cost, ST Disability = short-term disability, LT Disability = Long-term disability, QoL = quality of life.

doi.org/10.24095/hpcdp.28.3.02

3D plots can create problems

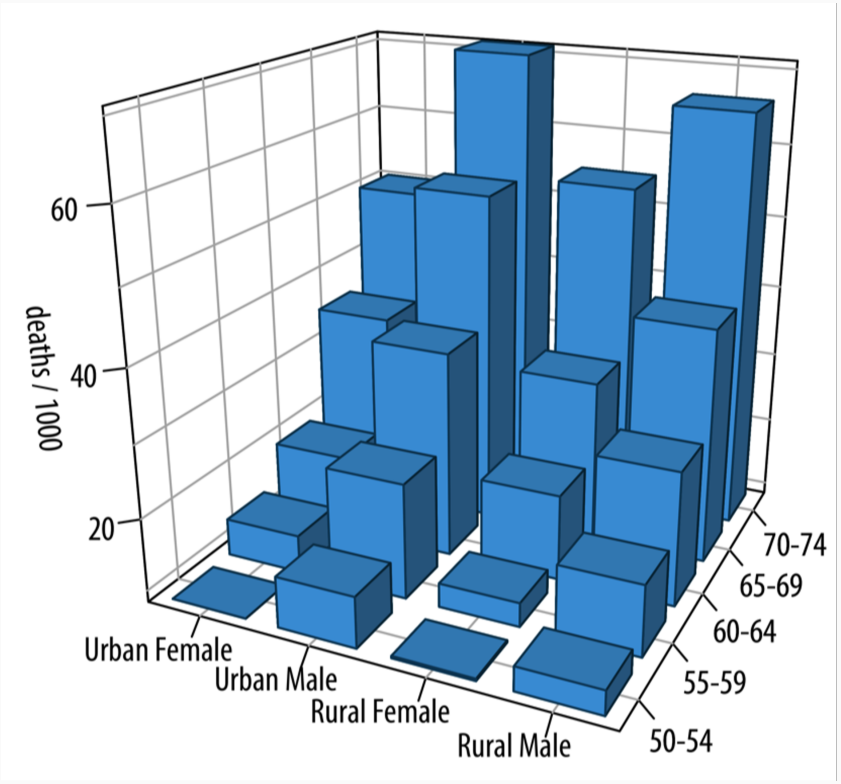
- Perspective can make values appear larger/smaller depending on the viewing angle.
- Overlapping objects can hide important data points.
- Axes become hard to read
- 3D plots on 2D screens distort depth and mislead the viewer.



Avoid 3D in general

Reading a plot should not require spatial reasoning.

- Consider faceting plots *per group*



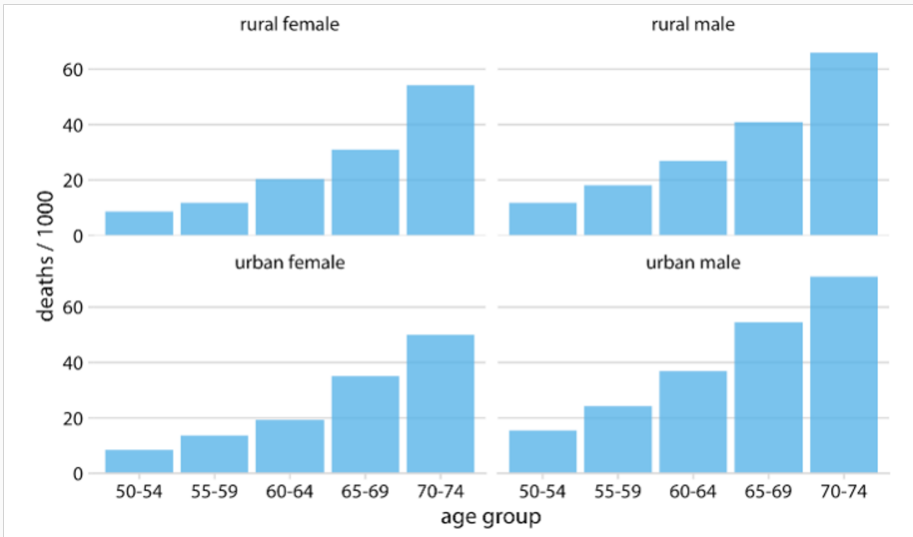
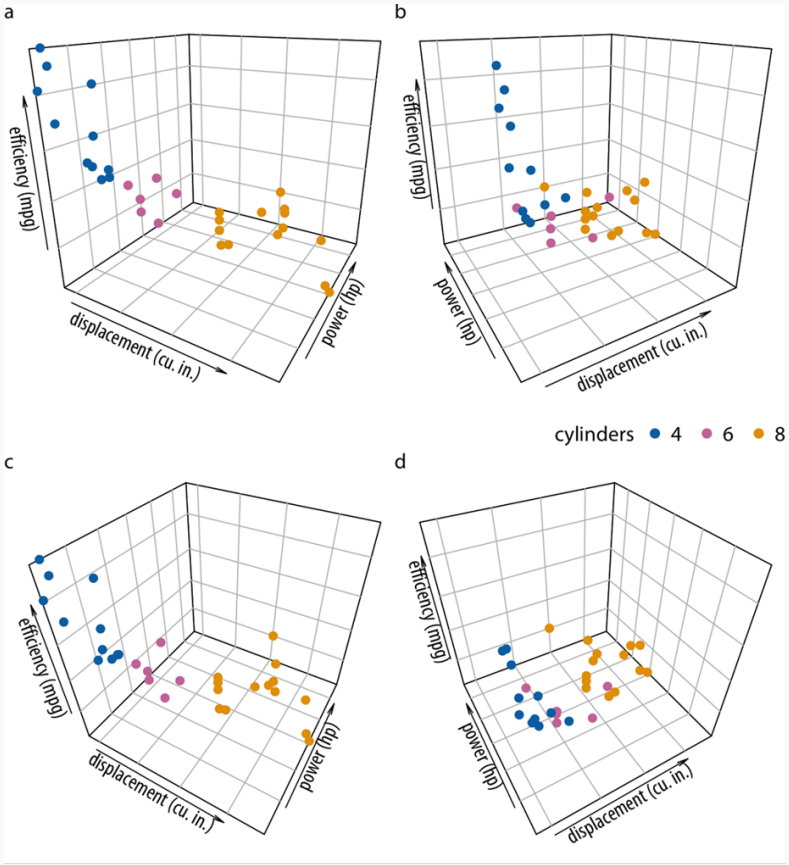


Figure 3: https://clauswilke.com/dataviz/no-3d.html

3D scatter plots

- Consider faceting plots *per variable/dimension*
- Consider using another *visual parameter* (e.g., size)



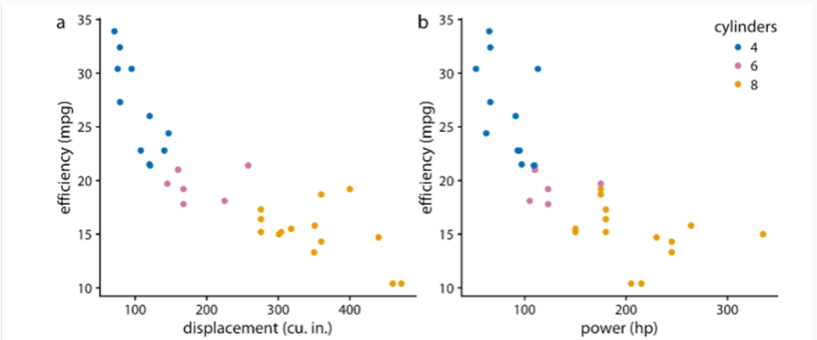


Figure 26.5: Fuel efficiency versus displacement (a) and power (b). Data source: *Motor Trend*, 1974.

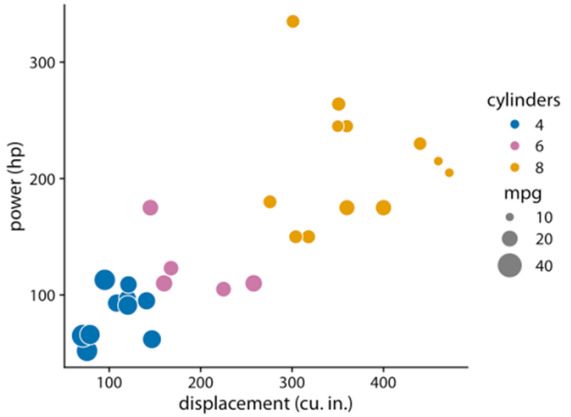
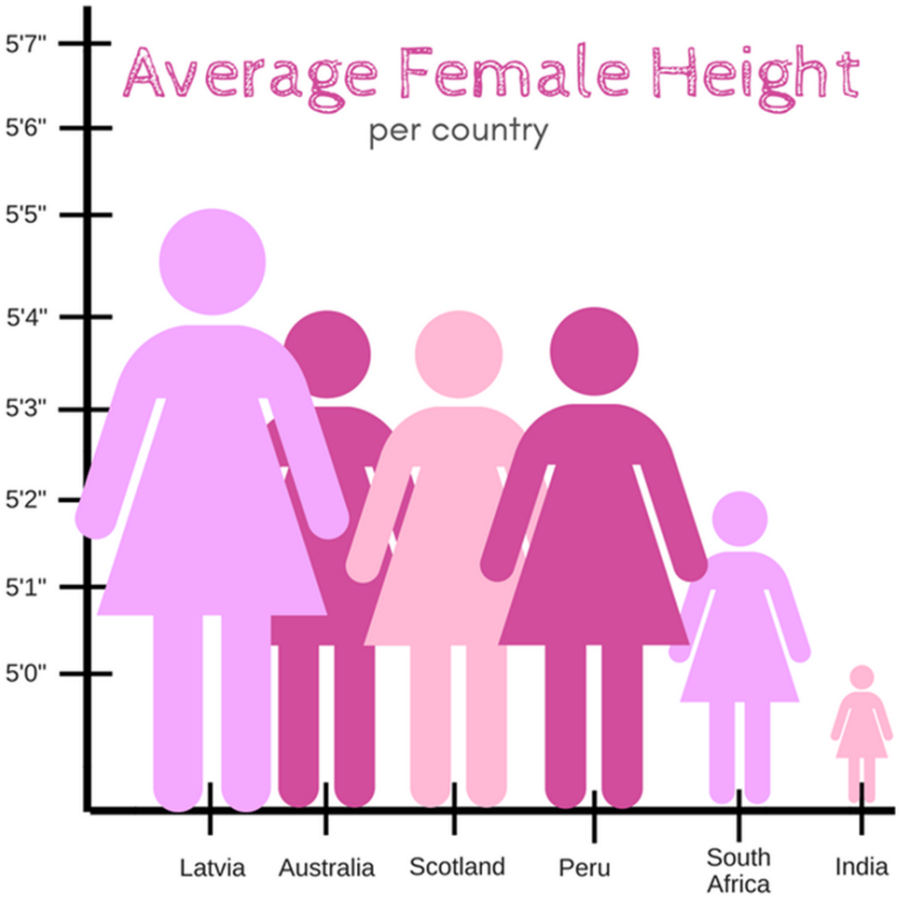


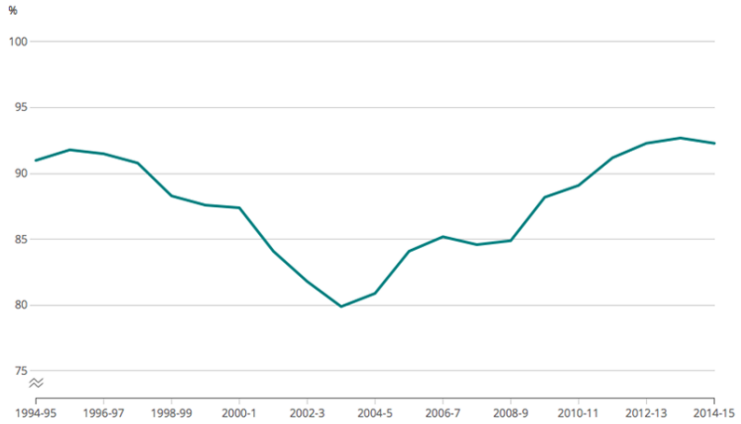
Figure 26.6: Power versus displacement for 32 cars, with fuel efficiency represented by dot size. Data source: *Motor Trend*, 1974.

Figure 4: https://clauswilke.com/dataviz/no-3d.html

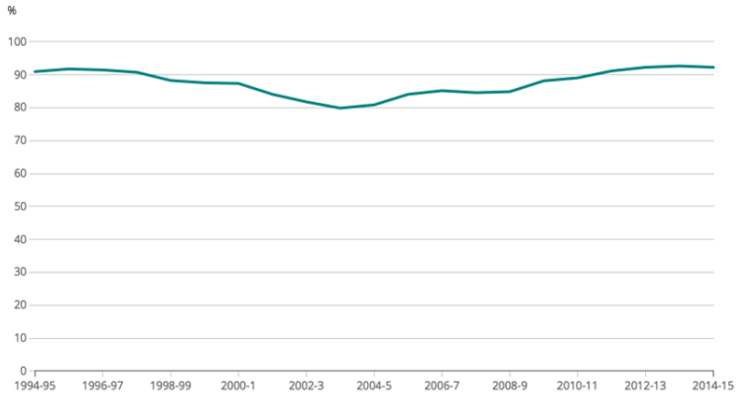
Range of the y-axis

Truncating the y-axis can be misleading and exaggerating trends (cf. this blog post). Because the visual exaggerates small differences.





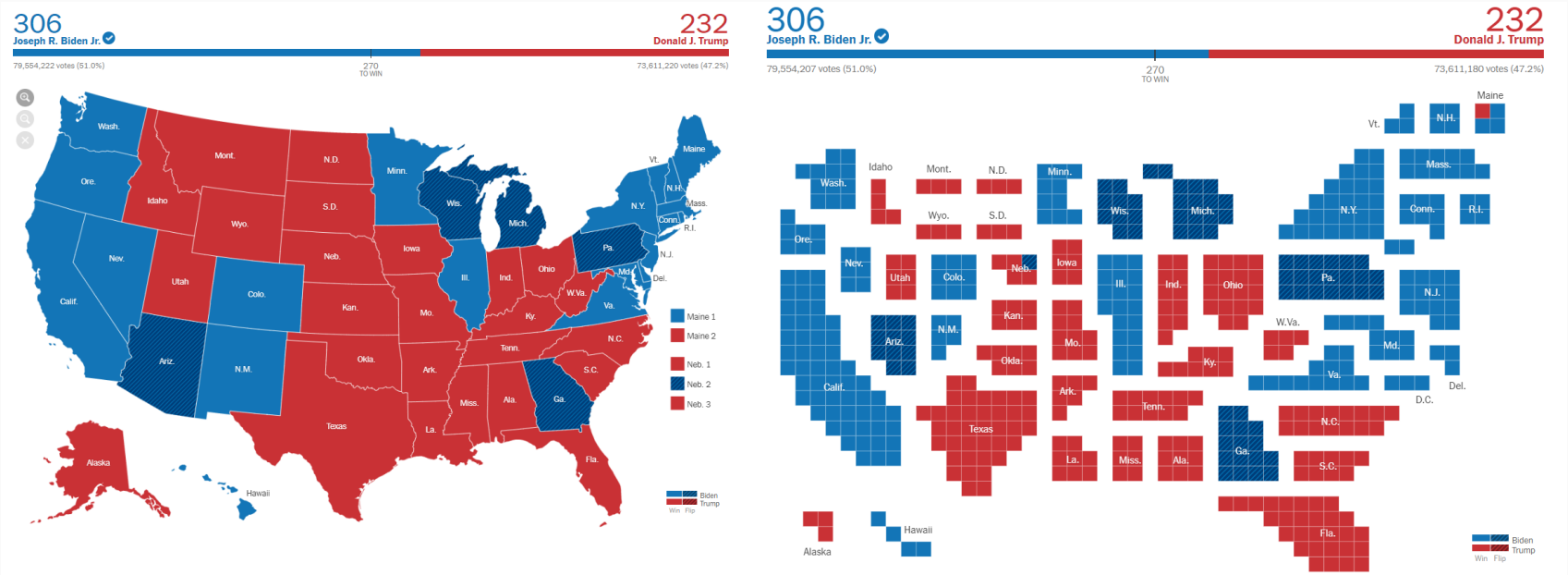
Source: NHS Immunisation Statistics - England, 2014-15, Table 8 and 9, HSCIC



Source: NHS Immunisation Statistics - England, 2014-15, Table 8 and 9, HSCIC

Rethink before plotting data on maps

- Rethink before mapping data. Coloring regions often misleads because maps show land area not people, exaggerate differences,hide variation, and distort perception.
- Left map: states keep their real geographic size, so large but less-populated states look visually dominant. Right map: states are resized into blocks based more closely on population or electoral votes.

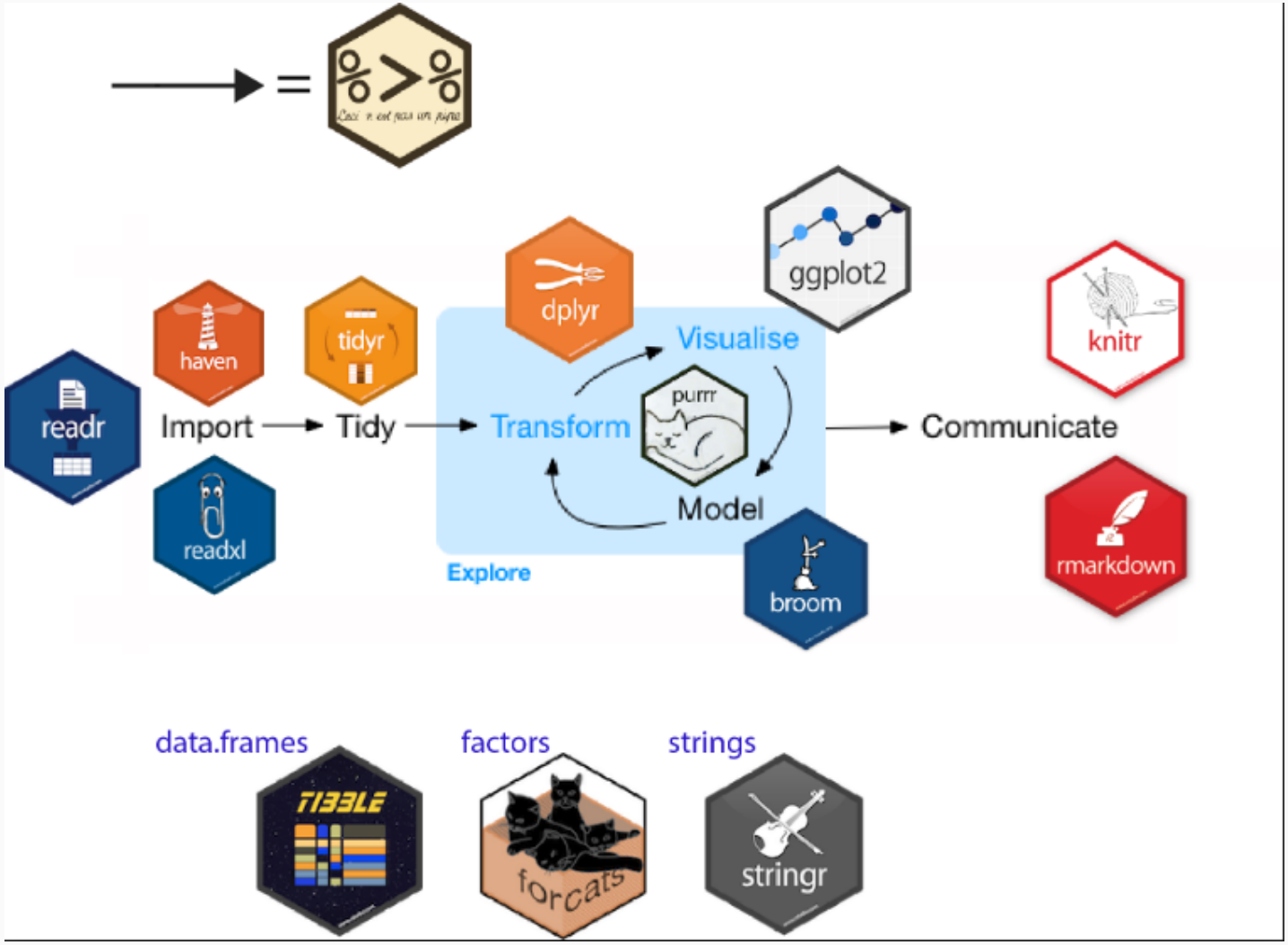


nytimes.com/2020/11/19/learning/whats-going-on-in-this-graph-2020-presidential-election-maps.html

Grammar of Graphics

R data science package: `tidyverse`

The `tidyverse` is a unified collection of R packages for importing, tidying, transforming, visualising, modelling, and communicating data.



‘tidyverse’

see https://tidyverse.org/packages/

Tidyverse packages

Contents

Installation and use

- Install all the packages in the tidyverse by running `install.packages("tidyverse")`.
- Run `library(tidyverse)` to load the core tidyverse and make it available in your current R session.

Learn more about the tidyverse package at https://tidyverse.tidyverse.org.

Core tidyverse

The core tidyverse includes the packages that you're likely to use in everyday data analyses. As of tidyverse 1.3.0, the following packages are included in the core tidyverse:

ggplot2

ggplot2 is a system for declaratively creating graphics, based on The Grammar of Graphics. You provide the data, tell ggplot2 how to map variables to aesthetics, what graphical primitives to use, and it takes care of the details. Go to docs...

dplyr

dplyr provides a grammar of data manipulation, providing a consistent set of verbs that solve the most common data manipulation challenges. Go to docs...

Grammar of Graphics: The `ggplot2` package

Think of a plot like a stack of transparent sheets placed on top of each other.

Describes all the non-data ink	Theme
Plotting space for the data	Coordinates
Statistical models & summaries	Statistics
Rows and columns of sub-plots	Facets
Shapes used to represent the data	Geometries
Scales onto which data is mapped	Aesthetics
The actual variables to be plotted	Data

Content of building blocks

Data	*{variables of interest}*				
Aesthetics	*x-axis* *y-axis*	*colour* *fill*	*size* *labels*	*alpha* *shape*	*line width* *line type*
Geometries	*point*	*line*	*histogram*	*bar*	*boxplot*
Facets	*columns*	*rows*			
Statistics	*binning*	*smoothing*	*descriptive*	*inferential*	
Coordinates	*cartesian*	*fixed*	*polar*	*limits*	
Themes	*non-data ink*				

Data and aesthetics

Data and aesthetics in `ggplot2`

- **Data** ('data')
  - A plot needs data. You tell ggplot what values to visualize. The dataset you are plotting and the variables of interest (e.g.columns)
- **Aesthetics** (`aes`)
  - They describe how data is represented visually.
  - They map variables in your data to graphical features. such as:
    - x-axis/y-axis (map variables to horizontal and vertical positions.)
    - colour(sets the outline colors of points/lines to distinguish groups/values.)
    - fill(sets the interior color of shapes like bars or boxes.)
    - size(controls the size of points or the thickness of lines.)
    - shape(determines the symbol used to represent points.)
    - alpha (adjusts how transparent or opaque plot elements appear.)
    - line type/width(changes the style of lines (e.g., solid) and line width.)
    - labels (adds or modifies text displayed on or around plot elements.)

The `ggplot()` function

The `data` and `aes` arguments of the function `ggplot()`:

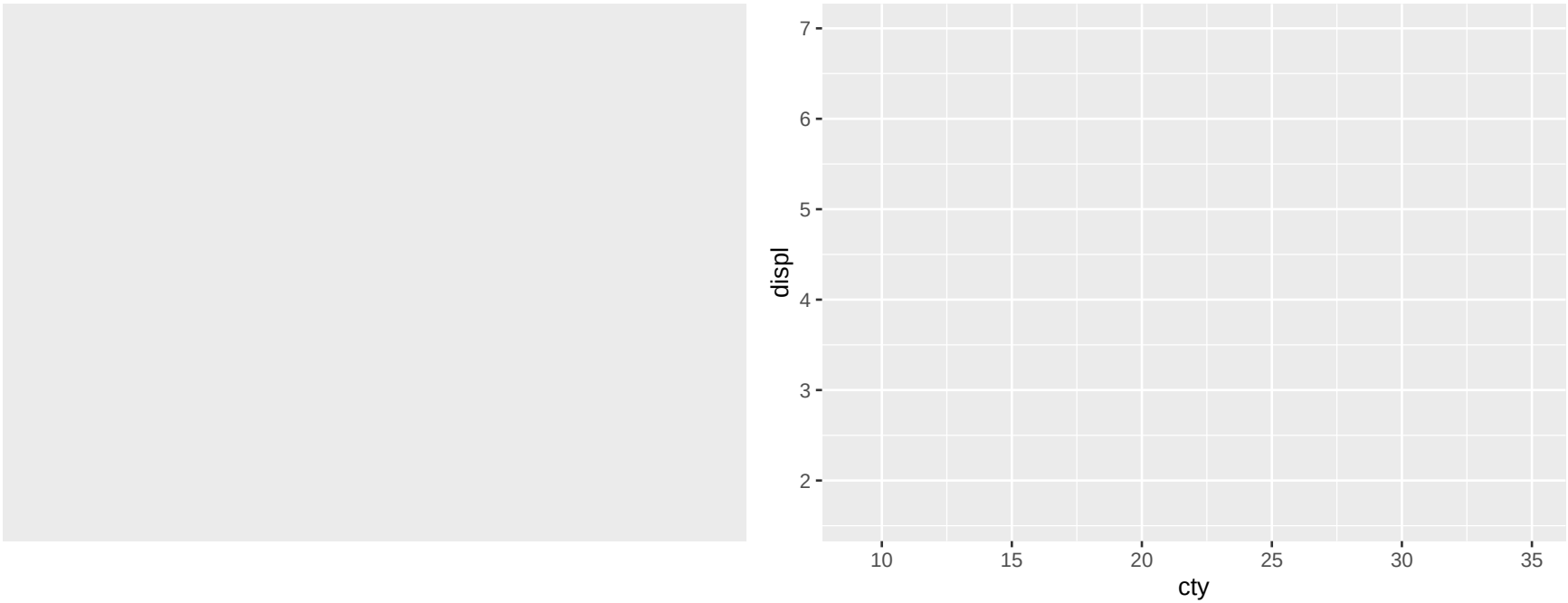
```
ggplot(data = <data>, mapping = aes(x = <var>, y = <var>, ...))
```

- `<data>`: name of the data set
- `mapping`: maps variables to aesthetics (axis, color, group, etc.)
- `aes`: the aesthetics
- More in the lab session

Example

- "`ggplot(mpg)`" creates an *empty* plot array for dataset "`mpg`"
- "`aes(x = cty, y = displ)`" maps the variable values to the axes
- cty = city fuel economy in miles per gallon
- displ = engine displacement in litres

```
library(patchwork) # patchwork '+' for displaying plots in a row
ggplot(mpg) +
ggplot(mpg, aes(x = cty, y = displ))
```



Geometrics in `ggplot2`

The layer(s) `geom_xxx()`

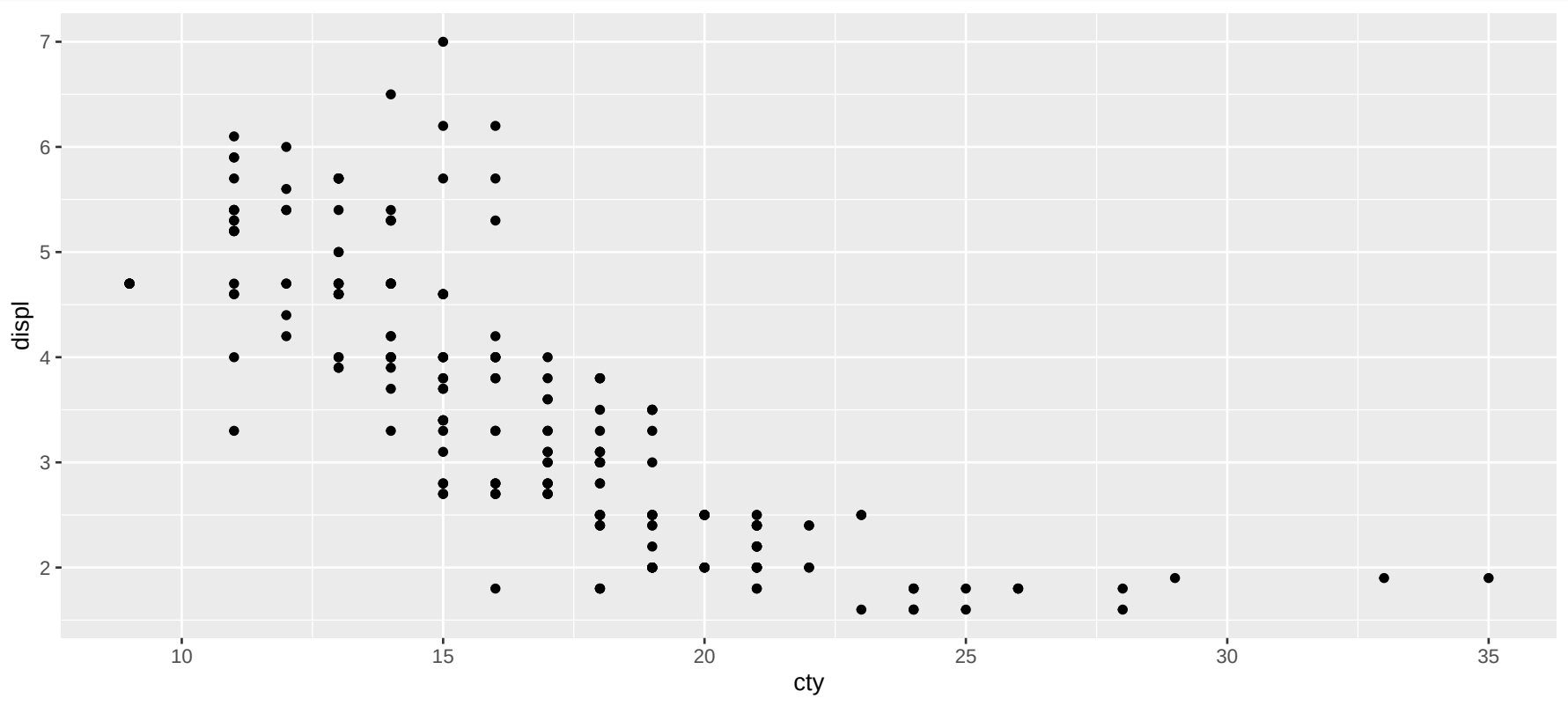
Geoms define shapes for representation (lines, points, bars, etc.)

- geoms are added with + sign
- multiple geoms can be added to same plot (e.g points and lines)
- within a geom other `aes()` arguments can be specified
- geoms have other arguments as well

Start a new line after each "+"

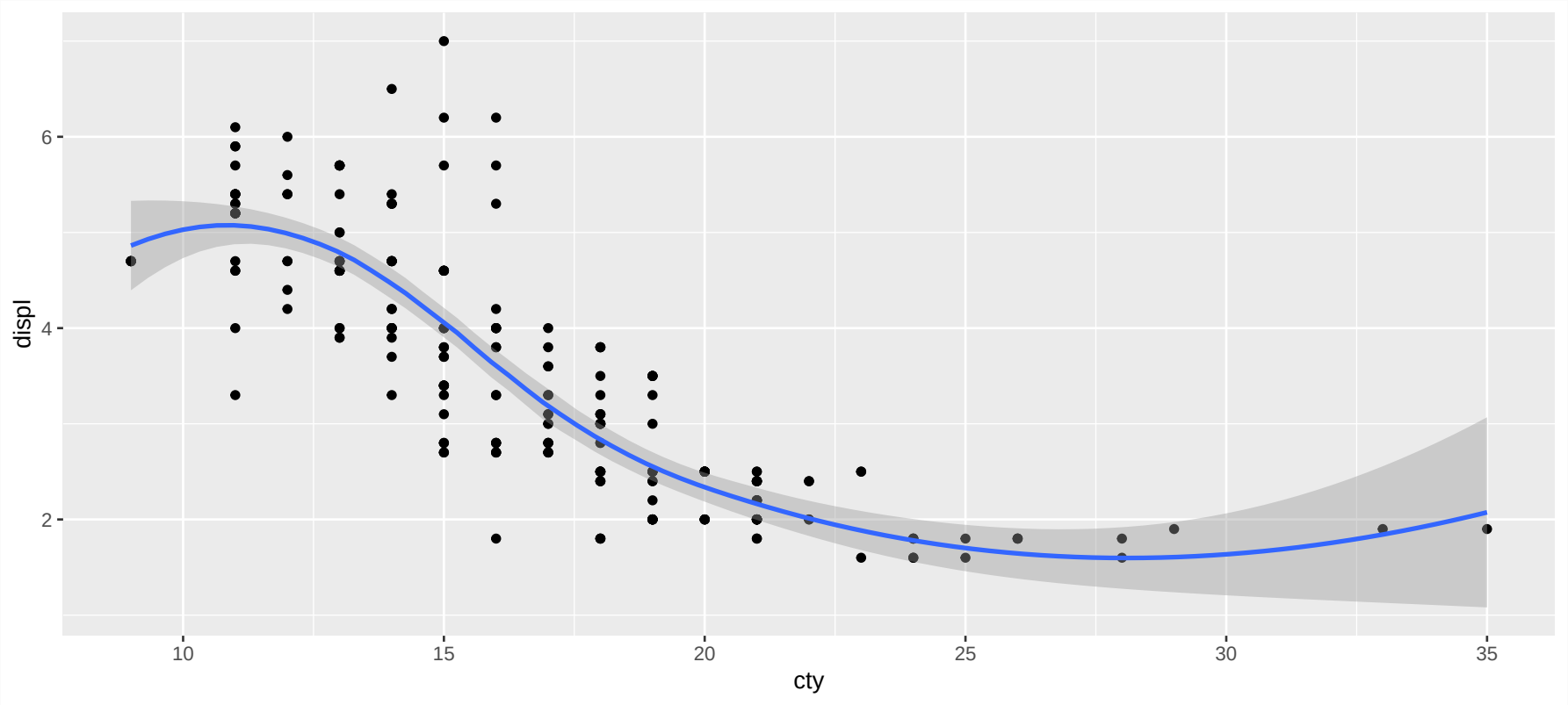
Example scatter plot

```
ggplot(mpg, aes(x = cty, y = displ)) +
  geom_point()        # adds points
```



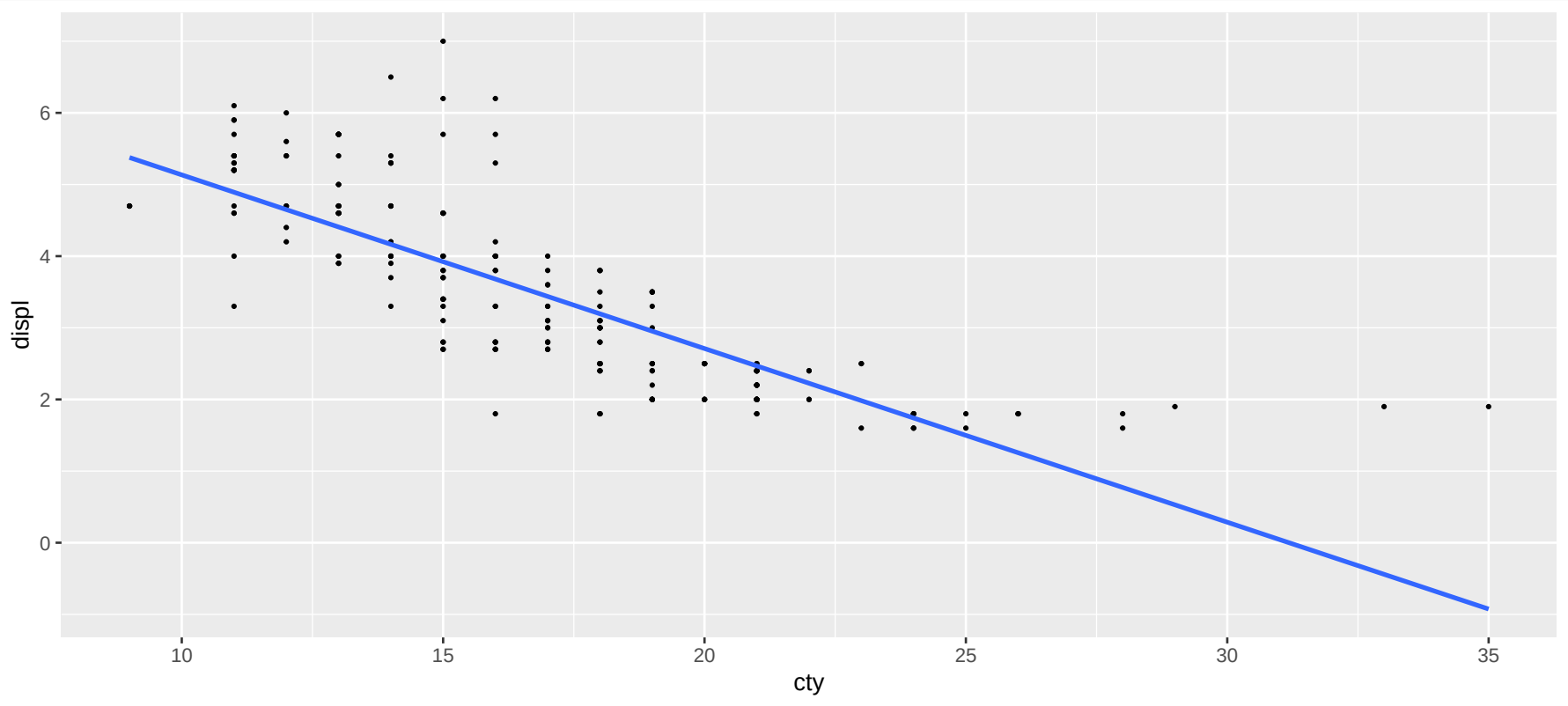
Example scatter plot

```
ggplot(mpg, aes(x = cty, y = displ)) +
  geom_point() +                          # adds points
  geom_smooth()                           # adds smooth regression line
```



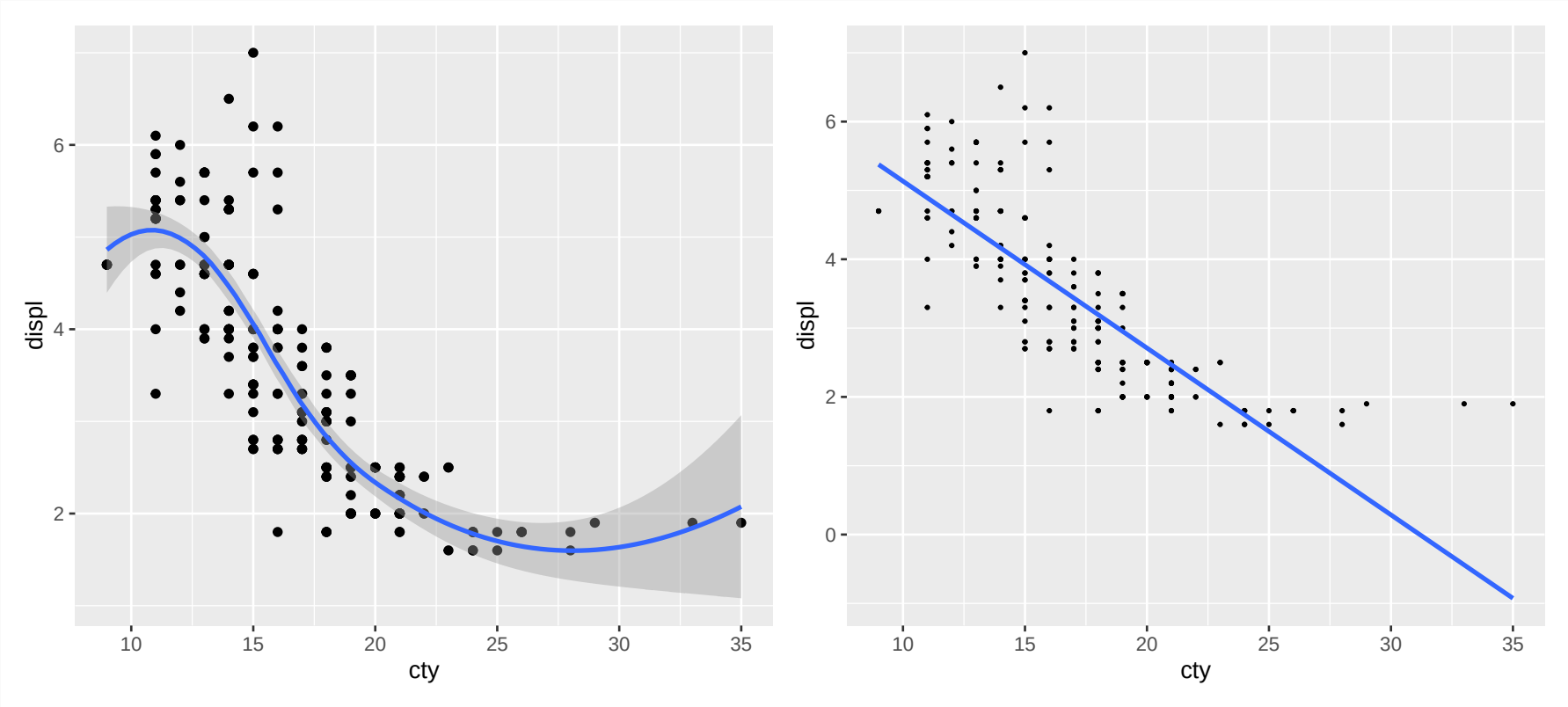
Example scatter plot

```
ggplot(mpg, aes(x = cty, y = displ)) +
  geom_point(size = .5) +                    # smaller points
  geom_smooth(method = "lm", se = FALSE)   # linear regression line
```



Example scatter plot

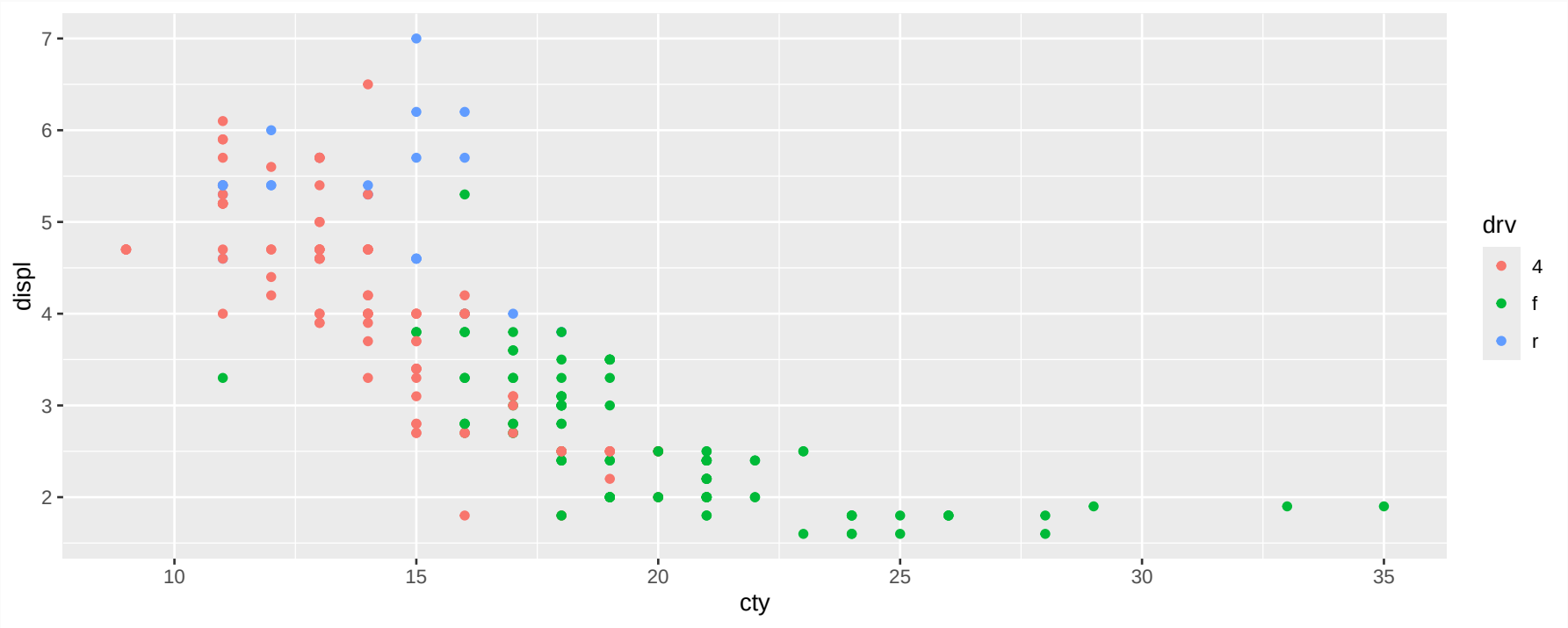
```
ggplot(mpg, aes(x = cty, y = displ)) +
  geom_point() +                          # adds points
  geom_smooth() +                         # adds smooth regression line

ggplot(mpg, aes(x = cty, y = displ)) +
  geom_point(size = .5) +                  # smaller points
  geom_smooth(method = "lm", se = FALSE)   # linear regression line
```



Examples `aes` in geoms

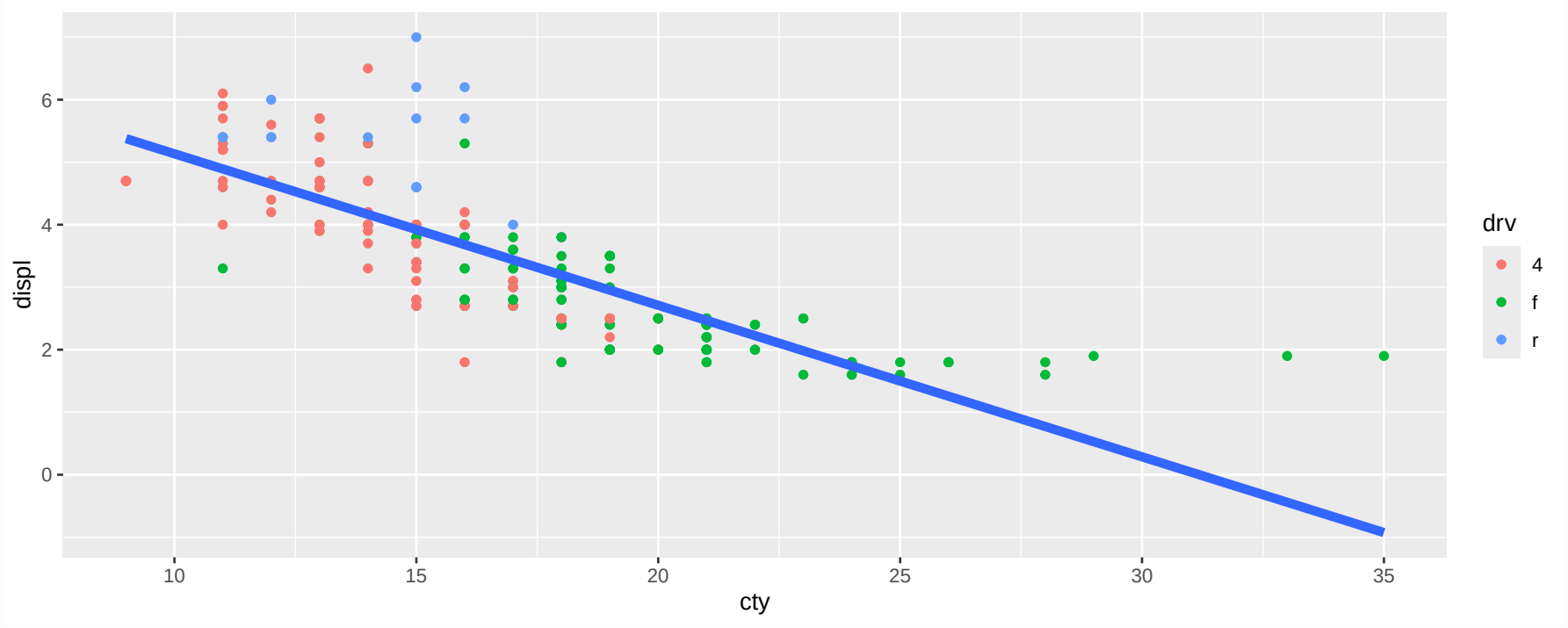
drv: drive type `f` is "front-wheel drive", `r` is "rear-wheel drive" and `4` is "four-wheel drive".

```
ggplot(mpg, aes(x = cty, y = displ)) +
  geom_point(aes(col= drv))    #  col as an argument in aes() inside geom
```



Examples aes in geoms

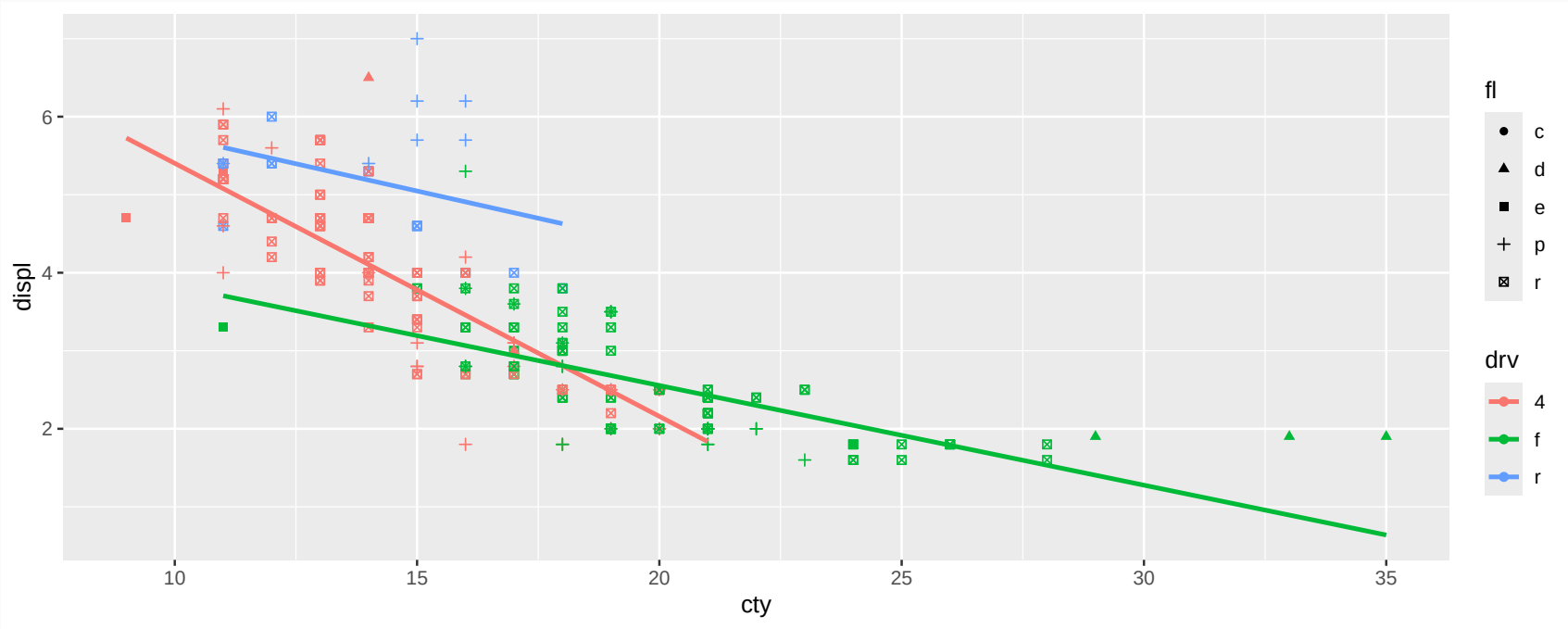
```
ggplot(mpg, aes(x = cty, y = displ)) +
  geom_point(aes(col= drv)) +  # col as an argument in aes() inside geom
  geom_smooth(method = "lm", se = FALSE, linewidth = 2) #linear regression line
```



Examples aes in geoms

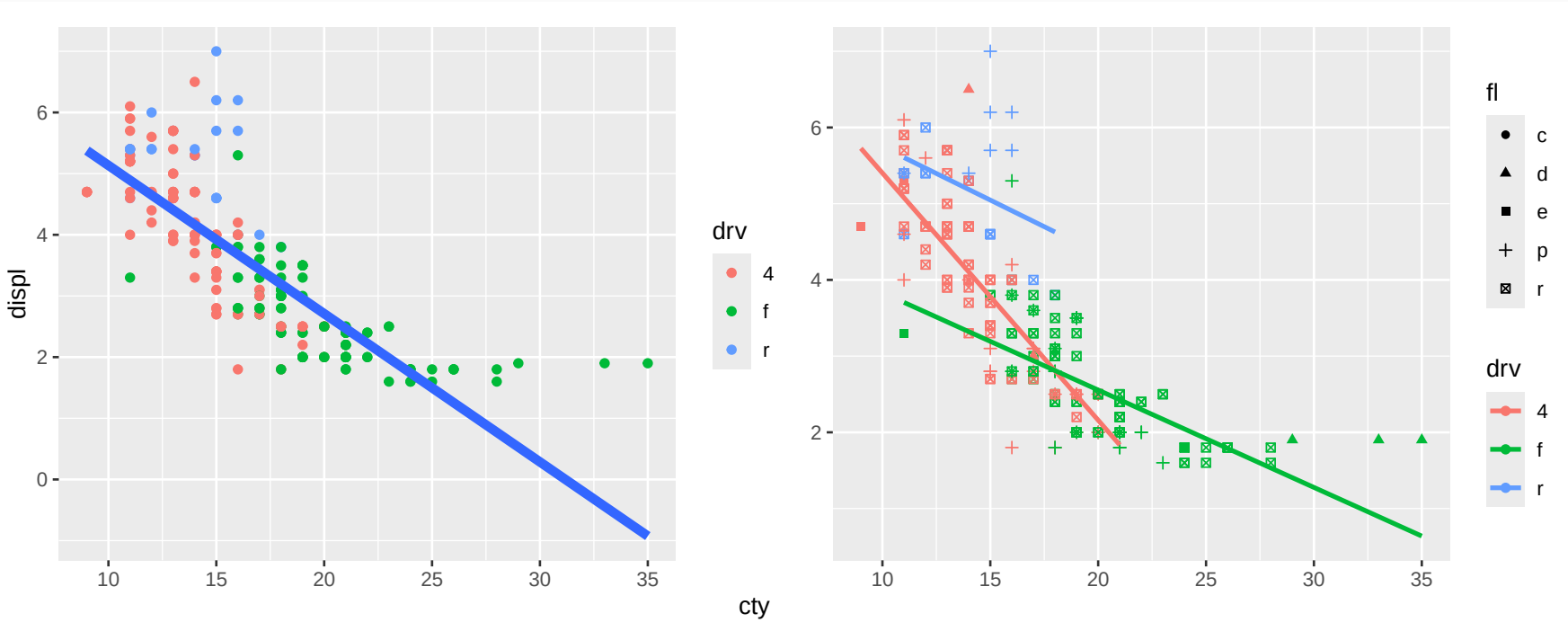
fl: fuel type.

```
ggplot(mpg, aes(x = cty, y = displ, col = drv)) +# col as an argument in aes() inside ggplot
  geom_point(aes(shape = fl)) +
  geom_smooth(method = "lm", se = FALSE)
```



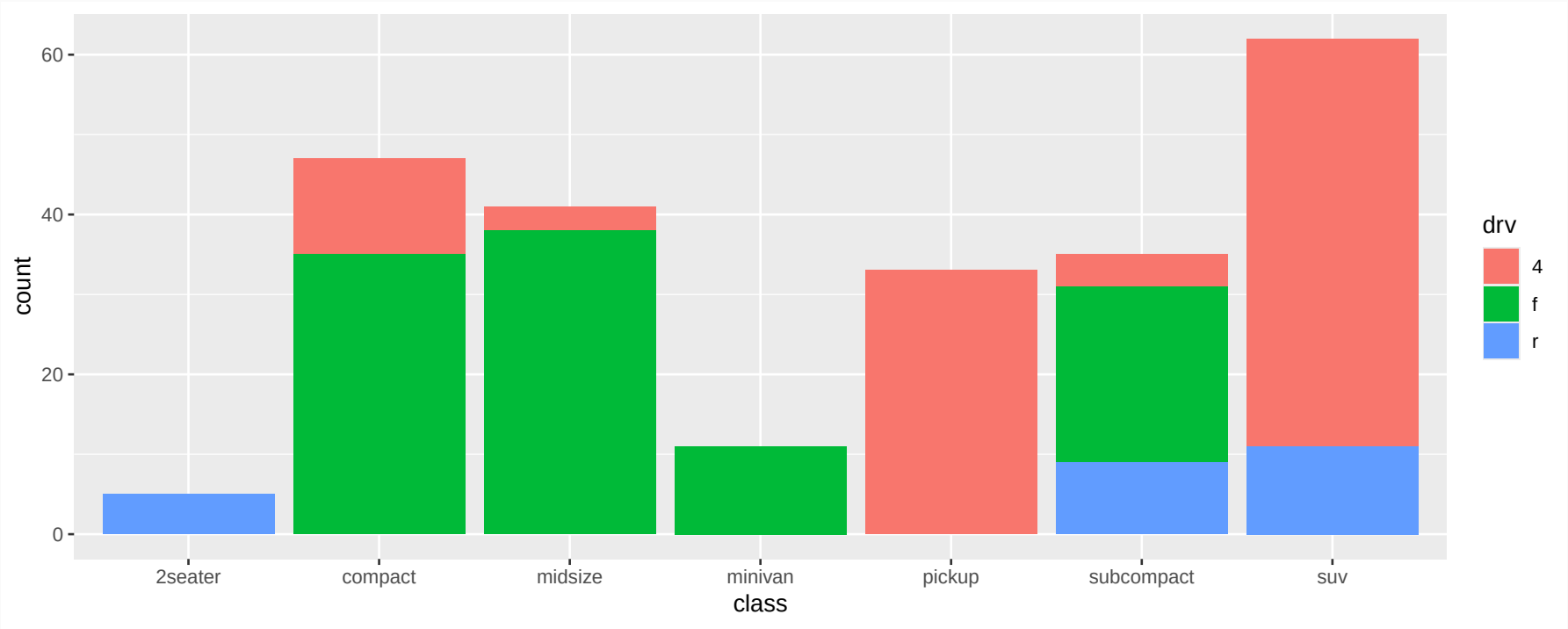
Examples aes in geoms

```
ggplot(mpg, aes(x = cty, y = displ)) +
  geom_point(aes(col= drv)) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 2) +

ggplot(mpg, aes(x = cty, y = displ, col = drv)) +
  geom_point(aes(shape = fl)) +
  geom_smooth(method = "lm", se = FALSE)  +

plot_layout(axes = "collect")                    # patchwork function
```



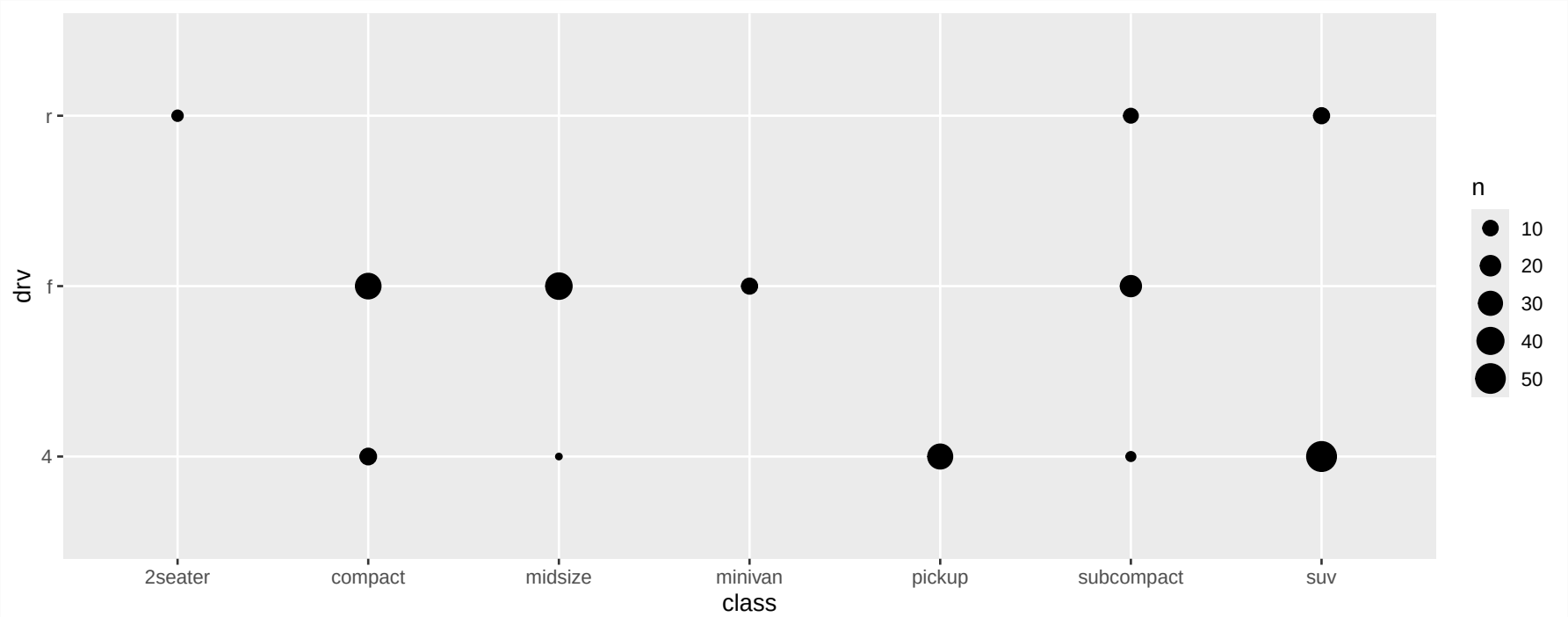
Categorical geoms: `geom_bar()` and `geom_count()`

```
ggplot(mpg) +
    geom_bar(aes(class, fill = drv)) # geom_bar() creates a bar chart, and counts the data
```



Categorical geoms: `geom_bar()` and `geom_count()`

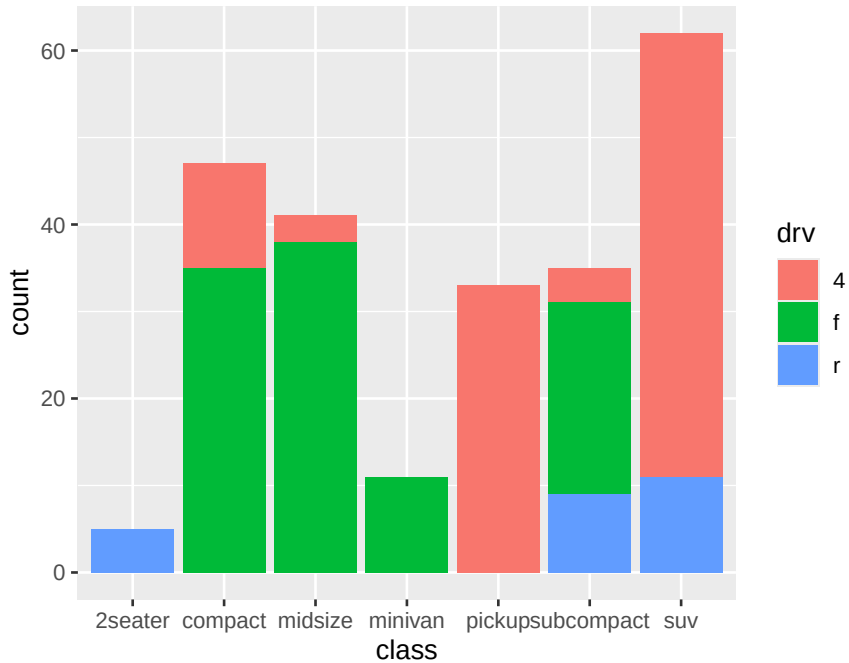
```
ggplot(mpg) +
    geom_count(aes(class, drv)) # geom_count() creates a bubble-style scatterplot
```

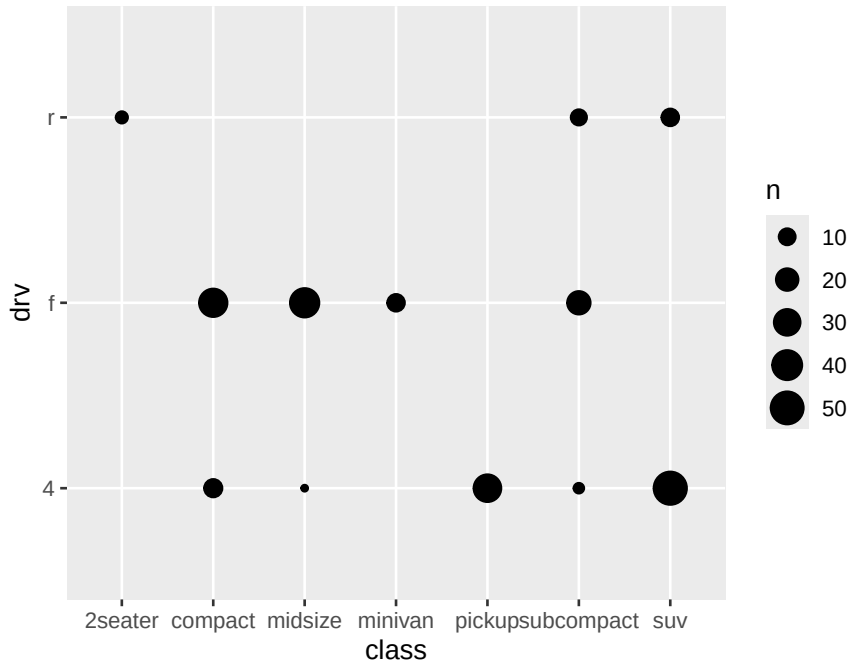



```
# geom_count() visualizes overlapping x-y cases by scaling point size. It shows how many cars
# Larger circles = more observations at the same coordinates.
# Useful for discrete/binned 2D data with overlaps.
```

Categorical geoms: `geom_bar()` and `geom_count()`

```
ggplot(mpg) +
    geom_bar(aes(class, fill = drv)) +

ggplot(mpg) +
    geom_count(aes(class, drv))
```

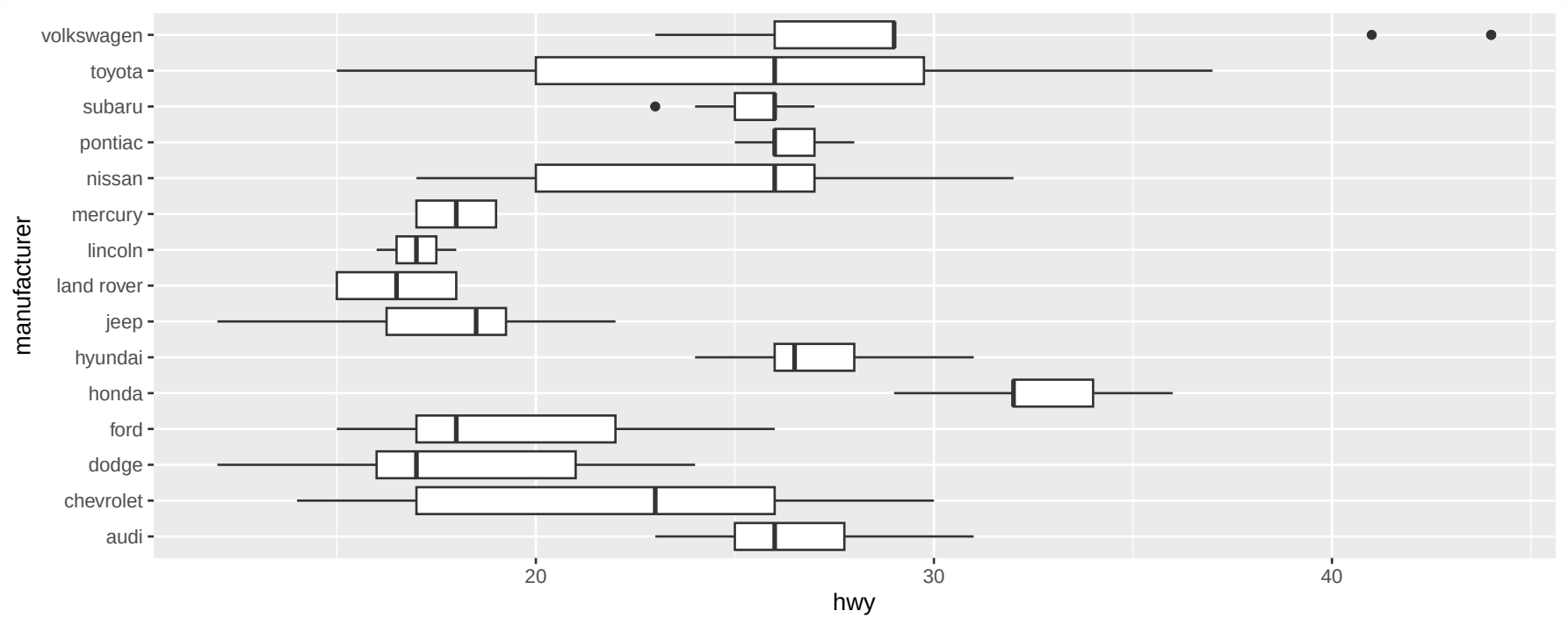




Numeric and categorical `geom_boxplot()` and `geom_density()`

hwy: highway fuel economy, measured in miles per gallon. manufacturer: the car company

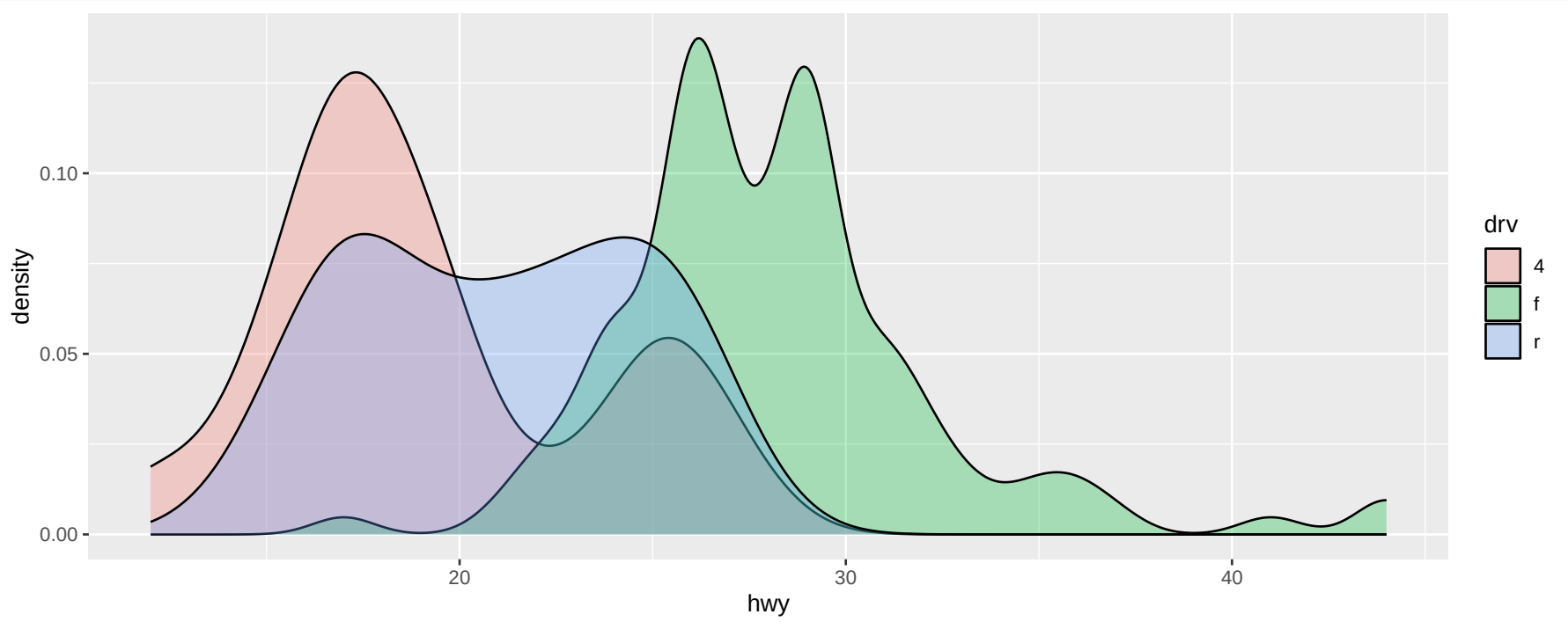
```
ggplot(mpg) +
  geom_boxplot(aes(hwy, manufacturer))
```




```
# geom_boxplot() creates boxplots to summarize distributions,
# using median,  quartiles, and outliers.
```

Numeric and categorical `geom_boxplot()` and `geom_density()`

```
ggplot(mpg) +
  geom_density(aes(hwy, fill = drv), alpha = .3)
```




```
# geom_density() plots a smooth estimate of a variable's distribution.
```

Overview geoms

There are many more “geoms”

- geom_point
- geom_bar
- geom_line
- geom_smooth
- geom_histogram
- geom_boxplot
- geom_violin
- geom_density
- geom_bar

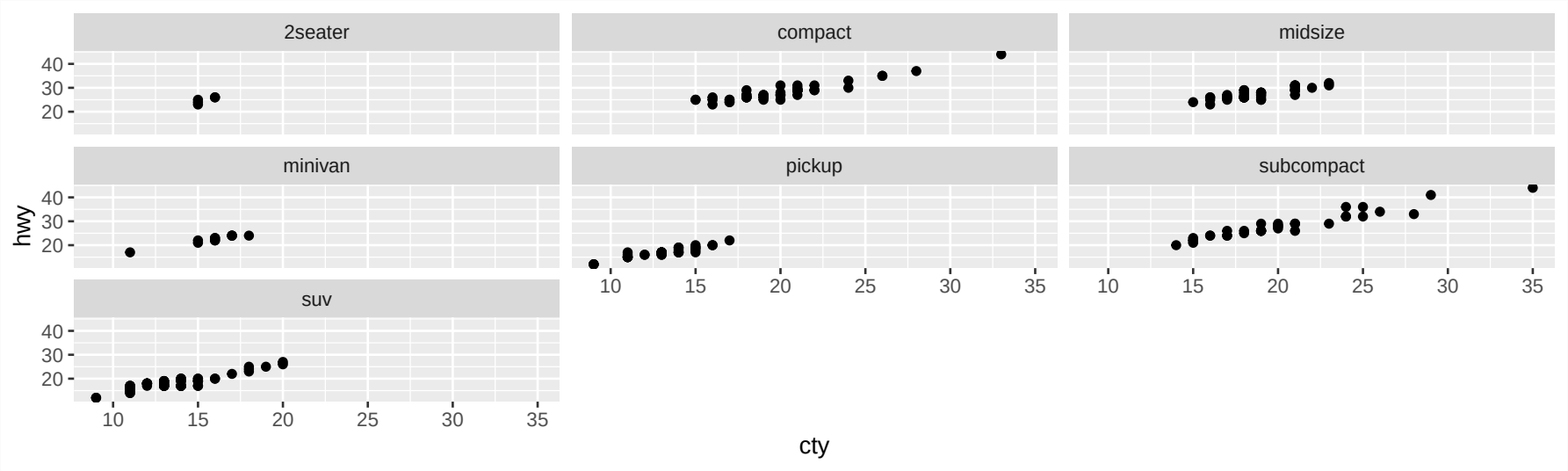
for more geoms and complete explanation see here:
https://ggplot2.tidyverse.org/reference/index.html#geoms

Facets

Split plots by categorical variable:`facet_wrap()`

- display multiples of `class`

```
ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point() +
  facet_wrap(vars(class))
```



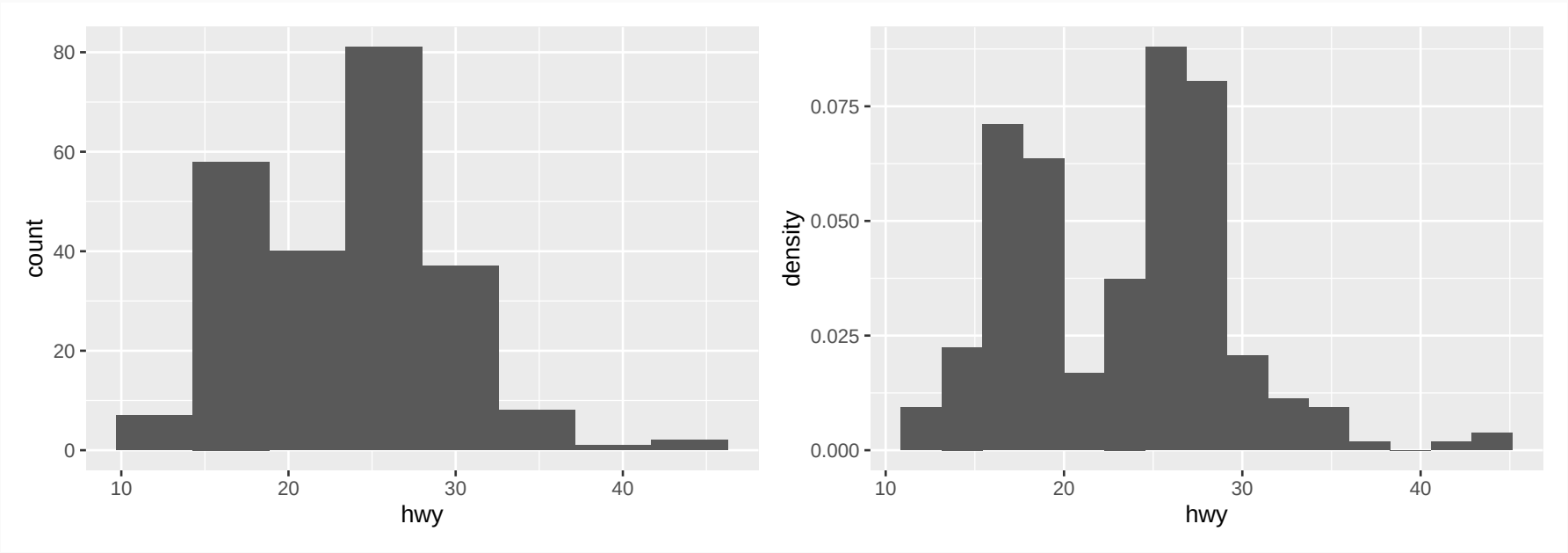

```
# facet_wrap() creates small multiple plots by splitting data into panels
# based on a grouping variable.
```

Statistics: Statistical transformations that ggplot2 applies before plotting the data.

Change summary statistic

- change number of bins and counts to density

```
ggplot(mpg, aes(hwy)) +
  geom_histogram(bins = 8) + # creating histogram with raw data

ggplot(mpg, aes(hwy)) +
  geom_histogram(aes(y = after_stat(density)), bins = 15) # density
```

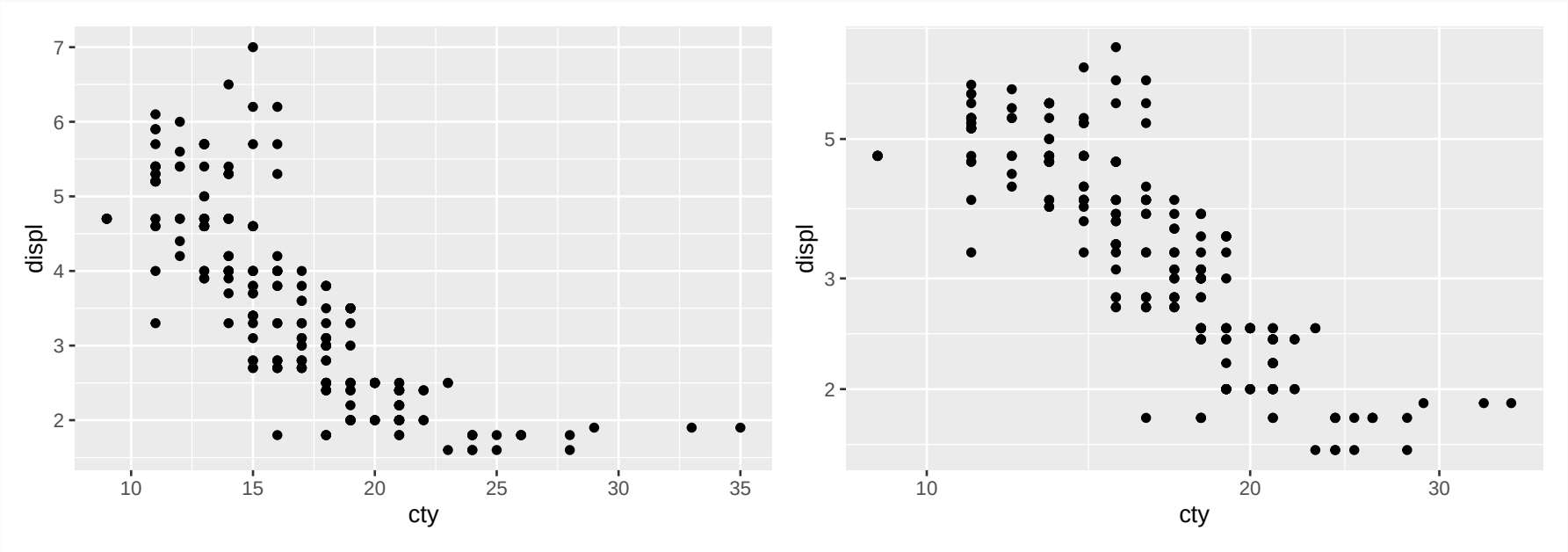


Scales: control the scaling and transformation of the axes (e.g., log scale).

Change axis scale

- display axes on log scale

```
ggplot(mpg, aes(cty, displ)) + geom_point() +

ggplot(mpg, aes(cty, displ)) + geom_point() +
  scale_x_log10() +
  scale_y_log10()
```



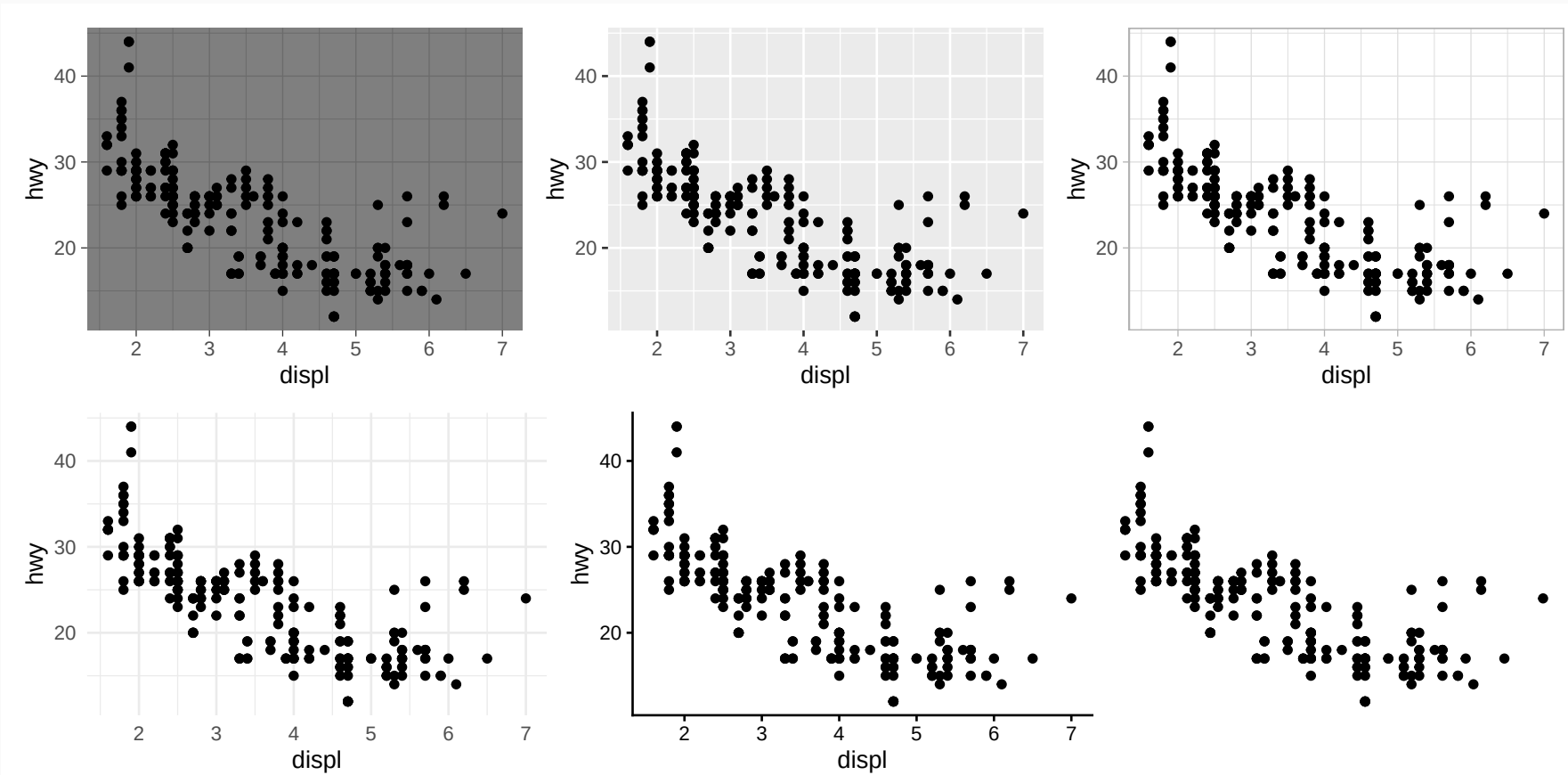

```
# The right plot uses a log10 scale, which stretches small values
# and compresses large ones, changing how the pattern appears.
```

Themes: change the visual style of a plot, backgrounds, gridlines, fonts, and layout,without changing the data.

Many to choose from

see here https://ggplot2.tidyverse.org/reference/ggtheme.html

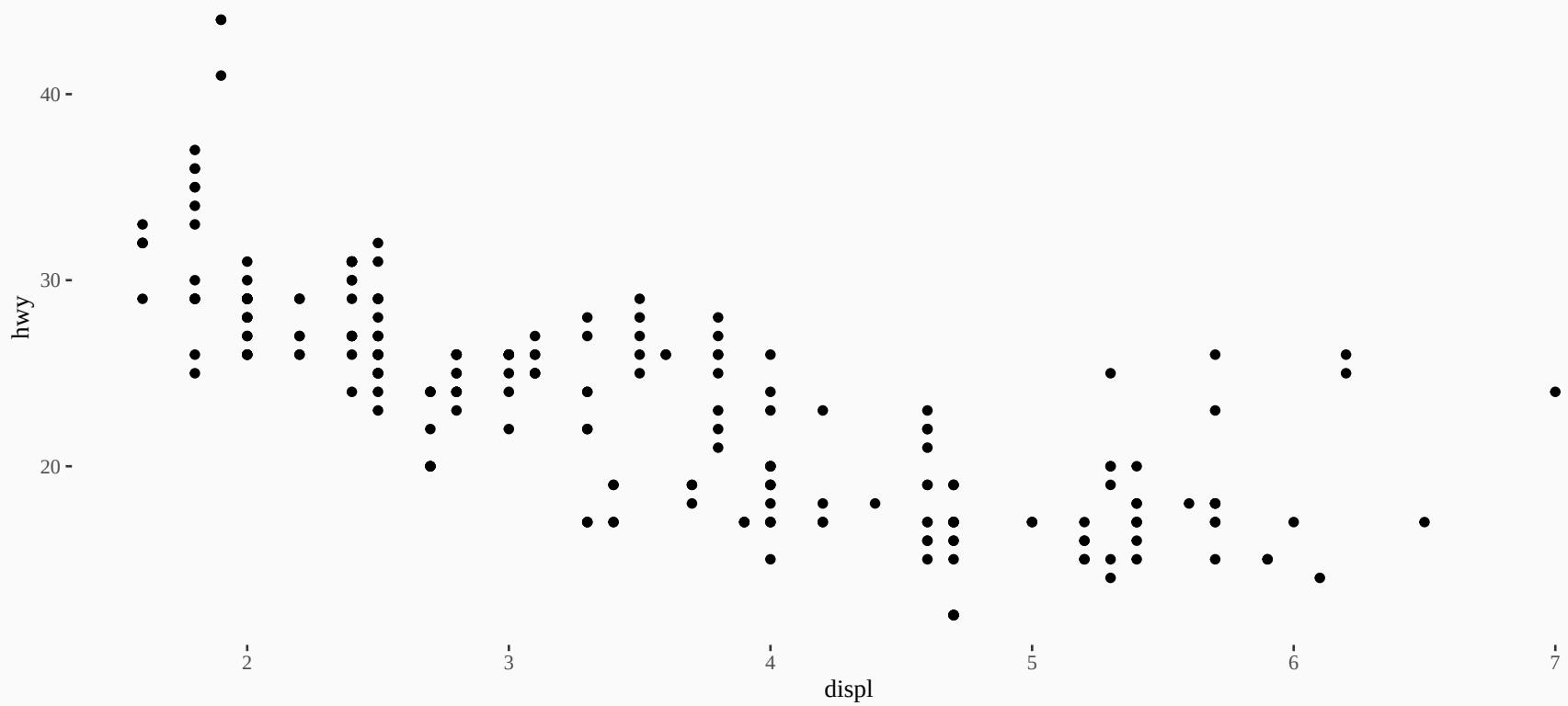
```
p <- ggplot(mpg) + geom_point(aes(displ, hwy))

(p + theme_dark()    + p + theme_gray()    + p + theme_light())  /
(p + theme_minimal() + p + theme_classic() + p + theme_void())
```



Theme closest to Tufte's principles `ggthemes::theme_tufte()`

- minimizes non-data ink and follows Tufte's principles of clean, uncluttered graphics.

```
#library(ggthemes)
 ggplot(mpg) + geom_point(aes(displ, hwy)) +
ggthemes::theme_tufte()
```



Literature

To learn `ggplot2`:

- ggplot2: Elegant Graphics for Data Analysis (3e) by Hadley Wickham

Additional resources:

- Garrick Aden-Buie's gentle - yet comprehensive - guide to 'ggplot2'
- Fundamentals of Data Visualization by Claus Wilke
- Cédric Scherer's blog
- /r/dataIsUgly on Reddit

Preview lab 1A

1. Make plots with `ggplot2`
2. Combine aesthetics, geoms, facets, themes

Make the exercises in the R Markdown template

- Open template in RStudio and read the instructions
- Save the file in an appropriate folder
- Insert R code in the R chunks
- Run the chunks to test for errors
- Knit the HTML file when the code is error free